

Tập luận văn đội tuyển quốc gia Trung Quốc năm 2020

Tập luận văn đội tuyển quốc gia Olympic Tin học

China Computer Federation, 2020

Nguồn gốc: Tập luận văn đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2020

IOI China candidate team papers / National training team 2020 collection.pdf

Tóm tắt

PDF nguồn là tập hợp luận văn đội tuyển quốc gia Trung Quốc năm 2020. Tập được tạo bằng XeTeX và có lớp văn bản Unicode đọc được. Bản LaTeX này chuyển ngữ phần nhận diện tập sách, toàn bộ mục lục, và bốn bài đầu tiên: bài của Jiang Mingrun về tối ưu các bài toán chia khối cổ điển bằng fractional cascading, bài của Chen Sunli về cây thống trị, và báo cáo ra đề *Ghép số* của Jiang Xunchi, cùng báo cáo ra đề *Khối liên thông nhỏ nhất* của Pan Junyue. Các hình nhỏ trong báo cáo thứ tư được dựng lại bằng TikZ từ bản render PDF.

1 Thông tin đầu sách

Tập sách có tiêu đề *Tập luận văn đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2020*. Tập PDF gồm 196 trang và được tạo bằng XeTeX. Trang bìa không ghi riêng tên huấn luyện viên trong phần văn bản trích xuất được.

2 Mục lục đã dịch

Trang	Bài viết	Tác giả
3	Bàn về tối ưu hóa bài toán chia khối cổ điển bằng fractional cascading	Jiang Mingrun
12	Bàn về cây thống trị và ứng dụng	Chen Sunli
22	Báo cáo ra đề <i>Ghép số</i>	Jiang Xunchi
32	Báo cáo ra đề <i>Khối liên thông nhỏ nhất</i>	Pan Junyue
42	Giới thiệu đơn giản về nguyên lý chuyển vị	Chen Yu
52	Bàn về duy trì động giá trị cực trị của hàm số	Li Baitian
63	Báo cáo ra đề <i>Trò chơi trên đồ thị</i>	Zuo Junchi
74	Một loại DFT không bình thường	Zhou Yuyang
87	Bài toán rừng hướng vào nhỏ nhất	Zhang Zheyu
102	Bàn về suffix automaton nén	Xu Yixuan
117	Bàn về nimber và thuật toán đa thức	Luo Yuxiang
133	Các bài toán liên quan tới định thức trong thi tin học	Wang Zhanpeng
151	Thuật toán hiệu quả để tính diện tích vùng ghép trên mặt cầu	Zhou Renfei
173	Bàn về ứng dụng đơn giản của lý thuyết nhóm trong thi tin học	Yu Haoxiang

3 Bàn về tối ưu hóa bài toán chia khối cổ điển bằng fractional cascading

Jiang Mingrun, Trường Trung học số 7 Thành Đô

Tóm tắt

Fractional cascading là một kỹ thuật tăng tốc tìm kiếm nhị phân một giá trị trong nhiều dãy số được tổ chức theo một cấu trúc nhất định. Bài viết này giới thiệu cách dùng thuật toán đó để tối ưu một bài toán chia khối cổ điển, đồng thời trình bày một vài ứng dụng của nó.

Lời nói đầu

Thuật toán fractional cascading đã ra đời hơn 20 năm, nhưng vẫn chưa được dùng rộng rãi trong cộng đồng thi thuật toán. Bài viết này thử dùng thuật toán đó để tối ưu bài toán chia khối cổ điển, với hy vọng đưa fractional cascading vào thi tin học. Mục 1 giới thiệu fractional cascading và ý nghĩa của nó; mục 2 giới thiệu bài toán chia khối cổ điển và các cách giải thường dùng; mục 3 dùng fractional cascading để tối ưu bài toán chia khối cổ điển, so sánh với các thuật toán truyền thống, đồng thời mở rộng bài toán cổ điển.

3.1 Fractional cascading

Mục này giới thiệu thuật toán fractional cascading. Ta sẽ dùng một ví dụ cụ thể để giải thích ý nghĩa của thuật toán.

3.1.1 Giới thiệu

Trong khoa học máy tính, fractional cascading là một kỹ thuật tăng tốc tìm kiếm nhị phân một số trong nhiều dãy số được tổ chức theo một cấu trúc nhất định: lần tìm kiếm đầu tiên là tìm kiếm nhị phân thông thường với độ phức tạp $O(\log n)$, còn các lần tìm kiếm nhị phân phía sau sẽ nhanh hơn lần đầu. Fractional cascading được Chazelle và Guibas đề xuất lần đầu trong hai bài báo năm 1986 [1, 2]. Đúng như tên gọi, “fractional” mang nghĩa rời rạc, nhỏ lẻ; “cascade” là một ẩn dụ chỉ sự xếp tầng như các thác nước nhỏ. Trong bài gốc, Chazelle và Guibas đưa ra một hình minh họa, ở đây gọi là Hình 1, mô tả rất trực quan ý nghĩa sâu hơn của tên gọi này: giống như thác nước từ cao xuống thấp, dần tách thành các nhánh nhỏ hơn rồi phủ chồng lên nhau theo từng tầng [3, 4].

Hình 1: hình minh họa do Chazelle và Guibas đưa ra trong bài gốc.

3.1.2 Minh họa bằng ví dụ

Ta sẽ mô tả thuật toán này dưới dạng một bài toán ví dụ.

Xét bài toán sau: cho k dãy có thứ tự với tổng độ dài là n , dãy thứ i ký hiệu là L_i . Cần xây dựng một cấu trúc dữ liệu hỗ trợ tìm kiếm nhị phân cùng một số q trong từng dãy trong k dãy, tức tìm vị trí của phần tử đầu tiên lớn hơn hoặc bằng q . Ví dụ dưới đây có $k = 4, n = 17$:

$$L_1 = 24, 64, 65, 80, 93,$$

$$L_2 = 23, 25, 26,$$

$$L_3 = 13, 44, 62, 66,$$

$$L_4 = 11, 35, 46, 79, 81.$$

Cách đơn giản nhất là lưu riêng từng dãy. Khi đó ta chỉ cần độ phức tạp không gian $O(n)$. Nhưng khi truy vấn, ta phải tìm kiếm nhị phân riêng trên từng dãy; trong trường hợp xấu nhất, khi k dãy có độ dài bằng nhau, độ phức tạp thời gian là $O(k \log(n/k))$.

Cách thứ hai hỗ trợ truy vấn nhanh hơn nhưng tốn nhiều không gian hơn: ta gộp k dãy này thành một dãy lớn L , và với mỗi phần tử trong L lưu kết quả tìm kiếm nhị phân của nó trong k dãy ban đầu. Nếu mỗi phần tử trong dãy sau khi gộp được viết là $x[a, b, c, d]$, trong đó x là giá trị, còn a, b, c, d là vị trí của phần tử kế tiếp tương ứng của x trong các dãy ban đầu, đánh số từ 0, thì:

$$\begin{aligned} L = & 11[0, 0, 0, 0], 13[0, 0, 0, 1], 23[0, 0, 1, 1], 24[0, 1, 1, 1], 25[1, 1, 1, 1], \\ & 26[1, 2, 1, 1], 35[1, 3, 1, 1], 44[1, 3, 1, 2], 46[1, 3, 2, 2], 62[1, 3, 2, 3], \\ & 64[1, 3, 3, 3], 65[2, 3, 3, 3], 66[3, 3, 3, 3], 79[3, 3, 4, 3], 80[3, 3, 4, 4], \\ & 81[4, 3, 4, 4], 93[4, 3, 4, 5]. \end{aligned}$$

Cách này đạt độ phức tạp truy vấn $O(k + \log n)$: chỉ cần tìm kiếm nhị phân trực tiếp trong L , rồi trả về thông tin đã lưu trong phần tử x . Tuy nhiên, nó cần không gian $O(nk)$.

Fractional cascading đồng thời đạt độ phức tạp thời gian và không gian tối ưu cho cùng bài toán: truy vấn $O(k + \log n)$, không gian $O(n)$. Ta xây dựng k dãy có thứ tự mới, dãy thứ i ký hiệu là M_i . Trong đó, dãy cuối cùng M_k giống L_k . Với mỗi dãy phía trước, M_i được tạo bằng cách trộn L_i với dãy con lấy mẫu cách một phần tử của M_{i+1} , bắt đầu từ phần tử thứ hai; với mỗi phần tử trong M_i , ta lưu kết quả tìm kiếm nhị phân của nó trong L_i và M_{i+1} . Với 4 dãy ở trên, ta có:

$$\begin{aligned} M_1 = & 24[0, 1], 25[1, 1], 35[1, 3], 64[1, 5], 65[2, 5], 79[3, 5], 80[3, 6], 93[4, 6], \\ M_2 = & 23[0, 1], 25[1, 1], 26[2, 1], 35[3, 1], 62[3, 3], 79[3, 5], \\ M_3 = & 13[0, 1], 35[1, 1], 44[1, 2], 62[2, 3], 66[3, 3], 79[4, 3], \\ M_4 = & 11[0, 0], 35[1, 0], 46[2, 0], 79[3, 0], 81[4, 0]. \end{aligned}$$

Nếu muốn truy vấn $q = 50$, trước hết ta tìm kiếm nhị phân trong M_1 và nhận được $64[1, 5]$. Số 1 nghĩa là kết quả tìm kiếm nhị phân trong L_1 là $L_1[1] = 64$; số 5 nghĩa là kết quả tìm kiếm nhị phân trong M_2 nằm xấp xỉ ở chỉ số 5. Chính xác hơn, nó là phần tử $79[3, 5]$ ở chỉ số 5 hoặc phần tử $62[3, 3]$ ngay trước đó. So sánh q với 62, ta biết kết quả đúng là $62[3, 3]$. Bằng cùng phương pháp, ta có thể nhận được kết quả tìm kiếm nhị phân của q trong L_2, M_3, L_3 , và M_4 .

Nói tổng quát hơn, với bất kỳ cấu trúc như vậy, ta có thể tìm kiếm nhị phân trước trong M_1 . Sau đó với mọi i , từ vị trí của q trong M_i ta định vị được vị trí của q trong L_i và trong M_{i+1} . Vị trí trong M_{i+1} mà kết quả truy vấn ở M_i trở tới hoặc chính là kết quả đúng của truy vấn trong M_{i+1} , hoặc chỉ lệch một vị trí, nên một phép so sánh là đủ để xác định. Tổng độ phức tạp thời gian là $O(k + \log n)$.

Trong ví dụ trên, tổng độ dài của bốn dãy mới là 25, không vượt quá $2n$. Nói chung, độ dài của M_i không vượt quá

$$|L_i| + \frac{1}{2}|L_{i+1}| + \frac{1}{4}|L_{i+2}| + \dots.$$

Tổng độ dài của mọi M_i không vượt quá

$$\sum |M_i| \leq \sum |L_i| \left(1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \dots \right) = 2n = O(n).$$

3.1.3 Trường hợp tổng quát

Xét một đồ thị có hướng không chu trình. Bậc vào và bậc ra của mỗi đỉnh đều không vượt quá một hằng số d , và trên mỗi đỉnh có lưu một dãy có thứ tự L_u . Một truy vấn cần tìm kiếm nhị phân một số q trong các dãy L_u được lưu trên mọi đỉnh u của một đường đi. Trong ví dụ phía trên, đồ thị có hướng đã cho là một dây chuyền độ dài 4.

Với mỗi đỉnh u , ta xây dựng một dãy mới M_u . Dãy M_u được tạo bằng cách trộn L_u với các dãy thu được khi lấy mẫu đều theo một tỉ lệ nhất định từ M_v của mọi hậu duệ trực tiếp v của u ; nếu muốn đạt không gian $O(n)$, tỉ lệ lấy mẫu phải nhỏ hơn $1/d$. Với mỗi phần tử, ta lưu kết quả tìm kiếm nhị phân của nó trong L_u và trong từng M_v . Trong ví dụ phía trên, $d = 1$ và tỉ lệ lấy mẫu là $1/2$.

Với một truy vấn, trước hết ta thực hiện một lần tìm kiếm nhị phân $O(\log n)$ trong dãy M_s được xây dựng ở điểm đầu s của đường đi. Giả sử ta đã biết kết quả tìm kiếm nhị phân của q trong M_u , và cần tìm kết quả tìm kiếm nhị phân của q trong M_v với một hậu duệ trực tiếp v của u . Nếu tỉ lệ lấy mẫu khi xây dựng cấu trúc dữ liệu là $1/r$, thì khoảng cách giữa vị trí trong M_v mà kết quả tìm kiếm nhị phân trong M_u trở tới và vị trí đúng trong M_v sẽ không vượt quá r . Vì r là hằng số, ta có thể tìm kết quả đúng trong M_v với độ phức tạp thời gian $O(1)$.

Nếu độ dài đường đi cần truy vấn là k , độ phức tạp thời gian của một truy vấn là $O(k + \log n)$.

Ngoài ra, fractional cascading có thể hỗ trợ chèn một phần tử vào một dãy trong $O(\log n)$, nhưng độ phức tạp truy vấn sẽ trở thành $O(\log n + k \log \log n)$. Điều này không liên quan nhiều tới bài viết này nên được lược bỏ.

3.2 Bài toán chia khối cổ điển

Mục này giới thiệu một bài toán chia khối kinh điển trong OI và cách giải truyền thống của nó. Ở mục sau ta sẽ thảo luận cách dùng fractional cascading để tối ưu bài toán này.

3.2.1 Mô tả bài toán

Cho một dãy độ dài n và n thao tác. Có hai loại thao tác:

1. Thao tác sửa: cho l, r, x ; với mọi $l \leq i \leq r$, tăng a_i thêm x .
2. Thao tác hỏi: cho l, r, x ; hỏi trong mọi $l \leq i \leq r$, có bao nhiêu chỉ số i thỏa $a_i \leq x$.

3.2.2 Cách giải thường dùng, thuật toán 1

Chia dãy thành các khối, mỗi khối gồm B phần tử liên tiếp, tổng cộng $O(n/B)$ khối. Trong mỗi khối, lưu kết quả sắp xếp của mọi phần tử trong khối. Như vậy với một đoạn $[l, r]$, nhiều nhất chỉ có 2 khối bị phủ không hoàn toàn; các khối còn lại hoặc được phủ hoàn toàn, hoặc không bị phủ.

Với thao tác sửa, các khối bị phủ không hoàn toàn cần được xây dựng lại trực tiếp. Lúc này phải tính lại kết quả sắp xếp. Nếu sắp xếp thô thì độ phức tạp là $O(B \log B)$; nhưng chú ý rằng với hai số đều không bị sửa hoặc hai số đều bị sửa, quan hệ lớn nhỏ giữa chúng không đổi. Vì vậy, nếu khi sắp xếp ta ghi lại vị trí của phần tử trước khi sắp xếp, ta có thể sắp riêng các số không bị sửa và các số bị sửa trong $O(B)$, rồi trộn lại, nên độ phức tạp thời gian là $O(B)$. Với khối được phủ hoàn toàn, ta có thể ghi một thể lười biểu thị rằng mọi phần tử trong khối đều được cộng cùng một số. Độ phức tạp cho mỗi khối là $O(1)$, tổng cộng có $O(n/B)$ khối, nên tổng độ phức tạp thời gian là $O(n/B)$.

Với thao tác hỏi, các khối bị phủ không hoàn toàn có thể được duyệt trực tiếp qua mọi phần tử, độ phức tạp $O(B)$. Với khối được phủ hoàn toàn, ta cần tìm kiếm nhị phân trên mảng đã sắp xếp, độ phức tạp cho mỗi khối là $O(\log B)$, tổng độ phức tạp thời gian là $O(n \log B/B)$.

Tóm lại, độ phức tạp thời gian cho một thao tác là $O(B + n \log B/B)$; tổng độ phức tạp thời gian là $O(nB + n^2 \log B/B)$. Khi lấy $B = \sqrt{n \log n}$, độ phức tạp thời gian là tối ưu và bằng $O(n\sqrt{n \log n})$.

3.2.3 Thuật toán offline, thuật toán 2

Mục này thảo luận thuật toán offline.

Trong bài này, thuật toán ở mục này chỉ dùng được với phạm vi đầu vào sau: mọi số trong đầu vào đều là số nguyên có trị tuyệt đối không vượt quá 2^ω , và $\omega = O(\log n)$.

Chú ý rằng nút thắt của thuật toán ở mục trước nằm ở tìm kiếm nhị phân, nên mục này xét cách tối ưu phần đó.

Tuy nhiên, tối ưu một lần tìm kiếm nhị phân đơn lẻ là việc khó; có thể dùng các cấu trúc dữ liệu như y-fast tree, nhưng không thực dụng. Do đó, để tối ưu thuật toán, ta cần tối ưu tổng độ phức tạp thời gian của nhiều lần tìm kiếm nhị phân. Có hai hướng: hướng thứ nhất là tối ưu nhiều truy vấn tìm kiếm nhị phân trong một khối; hướng thứ hai là tối ưu một truy vấn tìm kiếm nhị phân trên nhiều khối. Mục này thảo luận hướng thứ nhất, còn hướng thứ hai sẽ được thảo luận ở mục sau.

Xét việc lưu trước mọi truy vấn và xử lý từng khối. Khi xây dựng lại một khối, ta tính kết quả của mọi lần tìm kiếm nhị phân trước đó trên khối này. Khi đó bài toán chuyển thành thực hiện Q lần tìm kiếm nhị phân trong một dãy độ dài B .

Chú ý rằng nếu các số được tìm kiếm nhị phân tăng đơn điệu, ta có thể tận dụng tính đơn điệu để đạt độ phức tạp $O(B + Q)$. Vì vậy nếu có thể sắp xếp nhanh Q số, ta có thể nhanh chóng tính được kết quả của Q lần tìm kiếm nhị phân.

Khi $\omega = O(\log B)$, có thể dùng sắp xếp cơ số với cơ số B , độ phức tạp thời gian là

$$O\left(\frac{(B + Q)\omega}{\log B}\right) = O(B + Q).$$

Như vậy, độ phức tạp thời gian của bài toán chia khối ban đầu có thể được tối ưu thành $O(nB + n^2/B)$; lấy $B = \sqrt{n}$ là tối ưu, cho $O(n\sqrt{n})$.

3.3 Thuật toán dựa trên fractional cascading

Mục trước đã giới thiệu bài toán chia khối cổ điển trong OI và các cách giải truyền thống. Mục này thảo luận cách dùng fractional cascading để tối ưu bài toán đó.

Cách dùng sắp xếp cơ số có thể đạt độ phức tạp thời gian $O(n\sqrt{n})$, nhưng nó không còn dùng được nếu miền giá trị không phải số nguyên, hoặc nếu bài toán bị buộc phải online, nghĩa là trước khi đọc một thao tác mới thì phải tính xong kết quả của mọi truy vấn trước đó, nên không thể lưu trước toàn bộ truy vấn.

Một lần tìm kiếm nhị phân riêng lẻ rất khó tối ưu. Nhưng cần chú ý rằng thứ ta cần không phải một lần tìm kiếm nhị phân riêng lẻ, mà là tìm kiếm nhị phân cùng một số trên $O(n/B)$ dãy. Vì vậy, có thể xét dùng fractional cascading. Tuy nhiên, fractional cascading không hỗ trợ sửa trực tiếp, nên cần xử lý thêm.

3.3.1 Cách làm $O(n\sqrt{n} \log \log n)$, thuật toán 3

Cai Chengze của Đại học Thanh Hoa đã nghĩ ra thuật toán có độ phức tạp thời gian $O(n\sqrt{n} \log \log n)$ trước tác giả, nhưng chưa công bố chi tiết. Cách làm được mô tả trong mục này là một cách khá dễ mà tác giả suy đoán từ một số trao đổi liên quan; không rõ có trùng với cách làm của Cai hay không. Dù thế nào, xin cảm ơn Cai Chengze.

Trong thuật toán này, cách xây dựng fractional cascading là: trong mỗi khối, lấy mẫu đều $1/D$ số phần tử để xây dựng fractional cascading. Khi tìm kiếm nhị phân, nhờ fractional cascading ta định vị được một vị trí lệch không quá D , do đó trong mỗi khối riêng lẻ đạt độ phức tạp thời gian $O(\log D)$. Dùng cách này, cứ mỗi D khối liên tiếp xây dựng một nhóm fractional cascading; tổng cộng có $O(n/(BD))$ nhóm.

Khi thực hiện thao tác sửa, nhiều nhất chỉ có 2 nhóm fractional cascading bị phá vỡ cấu trúc; các nhóm khác nhiều nhất chỉ bị cộng cùng một số vào mọi phần tử, điều này không thay đổi cấu trúc fractional cascading. Vì mỗi khối chỉ lấy $O(B/D)$ phần tử để xây dựng fractional cascading, và mỗi nhóm fractional cascading chỉ được xây dựng giữa D khối, nên độ phức tạp thời gian để xây dựng lại một nhóm là $O(B)$.

Với thao tác hỏi, cần thực hiện một truy vấn trong mỗi nhóm fractional cascading. Độ phức tạp truy vấn trong một nhóm là $O(D + \log B)$; có tổng cộng $O(n/(BD))$ nhóm, nên tổng độ phức tạp là $O(n/B + n \log B/(BD))$. Ngoài ra, với mỗi khối còn cần một lần tìm kiếm nhị phân $O(\log D)$, tổng độ phức tạp là $O(n \log D/B)$. Ta cũng phải duyệt $O(B)$ phần tử trong các khối bị phủ không hoàn toàn.

Tóm lại, độ phức tạp thời gian của một thao tác có thể được tối ưu thành

$$O\left(B + \frac{n \log D}{B} + \frac{n \log B}{BD}\right),$$

tổng độ phức tạp thời gian là

$$O\left(nB + \frac{n^2 \log D}{B} + \frac{n^2 \log B}{BD}\right).$$

Lấy $B = \sqrt{n \log \log n}$, $D = \log n$, độ phức tạp thời gian là $O(n\sqrt{n \log \log n})$.

3.3.2 Cách làm $O(n\sqrt{n})$, thuật toán 4

Nếu dùng cách gộp ở mục 1.2 thì đã khó tối ưu tiếp. Tiếp theo, mục này bắt đầu từ một cách gộp khác để tối ưu bài toán thêm một bước.

Xây dựng một cây đoạn có n/B lá; các lá của cây đoạn lần lượt lưu dãy đã sắp xếp được duy trì trong mỗi khối. Dãy lưu ở đỉnh trong được tạo bằng cách trộn các dãy thu được khi lấy mẫu đều theo một tỉ lệ nhất định từ dãy lưu ở hai con, và ghi lại với mỗi phần tử kết quả tìm kiếm nhị phân của nó trong hai dãy lưu ở hai con. Nói cách khác, trong mục 1.3, cho mỗi đỉnh trong của cây đoạn nối tới hai con của nó để tạo đồ thị có hướng không chu trình; L_u ở lá là dãy được duy trì trong mỗi khối, còn L_u ở đỉnh trong là dãy rỗng, từ đó nhận được cấu trúc fractional cascading.

Khi thực hiện thao tác sửa, theo tính chất của cây đoạn, chỉ có $O(\log(n/B))$ đỉnh có đoạn tương ứng bị phủ không hoàn toàn. Nếu đoạn ứng với một đỉnh được phủ hoàn toàn, mọi phần tử trong cây con của đỉnh đó chỉ bị cộng thêm cùng một số, còn cấu trúc không đổi. Vì vậy chỉ có $O(\log(n/B))$ đỉnh cần tính lại dãy lưu trữ. Khi tỉ lệ lấy mẫu là $1/2$, độ dài dãy lưu ở mỗi đỉnh đều là $O(B)$, tổng độ phức tạp thời gian là $O(B \log(n/B))$, chưa đủ tốt. Nhưng nếu dùng tỉ lệ nhỏ hơn, chẳng hạn $1/3$, thì ở mỗi tầng của cây đoạn, độ dài dãy lưu ở mỗi đỉnh bằng $2/3$ so với tầng dưới; theo tính chất của cây đoạn, ở mỗi tầng chỉ có $O(1)$ đỉnh bị phủ không hoàn toàn. Vì vậy tổng độ phức tạp thời gian không vượt quá

$$O\left(B\left(1 + \frac{2}{3} + \frac{4}{9} + \frac{8}{27} + \dots\right)\right) = O(B).$$

Với thao tác hỏi, tương tự mục 1.3, ta có thể tìm kiếm nhị phân trước trong gốc, rồi từ kết quả tìm kiếm nhị phân trong một đỉnh xác định trong $O(1)$ kết quả tìm kiếm nhị phân ở hai con. Tổng độ phức tạp thời gian là $O(n/B + \log n)$. Ngoài ra, ta vẫn cần duyệt $O(B)$ phần tử trong các khối bị phủ không hoàn toàn.

Tóm lại, độ phức tạp thời gian của một thao tác là $O(B + n/B + \log n)$; tổng độ phức tạp thời gian là $O(nB + n^2/B + n \log n)$. Lấy $B = \sqrt{n}$, độ phức tạp thời gian là tối ưu và bằng $O(n\sqrt{n})$.

3.3.3 Tổng kết thuật toán

Tổng hợp lại, độ phức tạp thời gian của các thuật toán được nêu trong Bảng 1, trong đó thuật toán 1 là thuật toán truyền thống. Thuật toán 2 cũng đạt mục tiêu tối ưu, nhưng chỉ phù hợp với một phần phạm vi

đầu vào. Thuật toán 3 và thuật toán 4 đều là các thuật toán tối ưu bằng fractional cascading, không bị hạn chế khi sử dụng; trong các thuật toán đã biết, thuật toán 4 là tối ưu nhất.

Thuật toán	Độ phức tạp thời gian	Ghi chú
Thuật toán 1	$O(n\sqrt{n \log n})$	Chỉ phù hợp với một phần phạm vi đầu vào
Thuật toán 2	$O(n\sqrt{n})$	
Thuật toán 3	$O(n\sqrt{n \log \log n})$	
Thuật toán 4	$O(n\sqrt{n})$	

Bảng 1: So sánh các thuật toán

3.3.4 Mở rộng ứng dụng thuật toán

Fractional cascading không chỉ có thể áp dụng cho bài toán cổ điển này, mà còn có thể mở rộng sang một số bài toán khác.

Bài toán 1. Cho một dãy độ dài n và n thao tác. Có hai loại thao tác:

1. Thao tác sửa: cho l, r, x ; với mọi $l \leq i \leq r$, tăng a_i thêm x .
2. Thao tác hỏi: cho l, r, x ; hỏi đoạn $[l, r]$ có bao nhiêu đoạn con $[l', r']$ thỏa mọi số trong $[l', r']$ đều không vượt quá x .¹

Chú ý rằng với hai đoạn $[l, mid]$ và $[mid + 1, r]$, nếu ta biết đáp án của chúng, ở đây “đáp án” bao gồm đáp án của thao tác hỏi, vị trí của số đầu tiên lớn hơn x trong đoạn, và vị trí của số cuối cùng lớn hơn x trong đoạn, thì ta có thể tính đáp án của đoạn $[l, r]$.

Chia mỗi B số liên tiếp thành một khối, tổng cộng $O(n/B)$ khối. Với một truy vấn, ta có thể tách đoạn $[l, r]$ thành $O(n/B)$ đoạn có độ dài không vượt quá B , trong đó chỉ có $O(1)$ đoạn không phải là nguyên một khối. Với đoạn không phải nguyên một khối, có thể tách tiếp thành $O(B)$ đoạn độ dài 1 để xử lý. Với một khối nguyên vẹn, ta tính dãy đã sắp xếp của mọi phần tử trong khối, và tính đáp án của khối khi từng phần tử trong khối được dùng làm x . Phần sau có thể được tính bằng cách sau:

Xét việc xử lý theo thứ tự tăng dần. Đưa mọi số lớn hơn x vào một danh sách liên kết theo thứ tự vị trí xuất hiện; để tiện xử lý, có thể thêm nút đặc biệt ở hai đầu danh sách. Khi x tăng dần, các phần tử trong danh sách sẽ dần bị xóa. Khi tính đáp án của khối, vị trí số đầu tiên lớn hơn x và vị trí số cuối cùng lớn hơn x trong đoạn có thể được lấy trực tiếp qua con trỏ của các nút đặc biệt. Khi tính đáp án thao tác hỏi, cần xét ảnh hưởng của việc xóa một nút lên đáp án: rõ ràng mọi đoạn chứa nút này nhưng không chứa tiền nhiệm và hậu nhiệm của nút này sẽ được cộng vào đáp án, và không có ảnh hưởng nào khác. Bước này có độ phức tạp thời gian $O(B)$, nên không làm tăng độ phức tạp thời gian xây dựng lại khối.

Sau khi tính được phần này, phần còn lại cơ bản giống cách giải bài toán chia khối cổ điển ở mục 2, và có thể được tối ưu bằng phương pháp ở mục 3.2.

Bài toán 2. Cho một dãy độ dài n và n thao tác. Có hai loại thao tác:

1. Thao tác sửa: cho l, r, x ; với mọi $l \leq i \leq r$, tăng a_i thêm x .
2. Thao tác hỏi: cho l, r, x ; hỏi trong mọi đoạn con của đoạn $[l, r]$, tổng các phần tử trong đoạn con lớn nhất là bao nhiêu.

¹Bài toán này được chỉnh sửa từ Comet OJ Contest #7 F: https://cometoj.com/contest/52/problem/F?problem_id=2426.

Trước hết trình bày cách làm của lời giải chính thức.

Tương tự bài toán trước, với hai đoạn $[l, mid]$ và $[mid + 1, r]$, nếu ta biết đáp án của chúng, ở đây “đáp án” bao gồm tổng đoạn con lớn nhất trong đoạn, tức đáp án của thao tác hỏi, tổng tiền tố lớn nhất trong đoạn, và tổng hậu tố lớn nhất trong đoạn, thì ta có thể tính đáp án của đoạn $[l, r]$.

Vấn xét chia mỗi B số liên tiếp thành một khối, tổng cộng $O(n/B)$ khối. Điểm mấu chốt là xử lý trường hợp một khối nguyên vẹn.

Để xử lý một khối nguyên vẹn, ta cần duy trì đáp án khi mọi số trong cả khối được cộng cùng một số. Trước hết xử lý tình huống này.

Xét dùng cây đoạn để duy trì. Với một đỉnh, cần duy trì đáp án của đoạn mà đỉnh đó đại diện khi mọi số trong đoạn được cộng cùng một số x . Không khó chứng minh đây là một hàm tuyến tính từng đoạn theo x , và số đoạn cùng bậc với độ dài của đoạn. Thông tin duy trì ở một đỉnh có thể được gộp từ hai con của nó; phép gộp chỉ cần cộng hai hàm từng đoạn hoặc lấy max, có thể hoàn thành trong $O(len)$, trong đó len là số đoạn của hàm từng đoạn, cùng bậc với độ dài đoạn. Như vậy, độ phức tạp thời gian xây dựng cây đoạn là tổng độ dài các đoạn tương ứng với mọi đỉnh trong cây đoạn, tức $O(B \log B)$.

Nhưng với bài toán ban đầu, làm như vậy sẽ gặp vài vấn đề. Vấn đề thứ nhất là khi thực hiện thao tác sửa, cần xây dựng lại các khối bị phủ không hoàn toàn. Nếu xây thô lại toàn bộ cây đoạn thì độ phức tạp thời gian là $O(B \log B)$, chưa đủ tốt.

Chú ý rằng khi xây dựng lại một khối, thao tác sửa trên khối đó chỉ là cộng đoạn. Theo tính chất của cây đoạn, ở mỗi tầng chỉ có $O(1)$ đỉnh bị phủ không hoàn toàn, còn sửa trên đỉnh được phủ hoàn toàn chỉ là tịnh tiến hàm từng đoạn, có thể dùng thẻ lười để duy trì. Vì vậy, ở mỗi tầng chỉ có $O(1)$ đỉnh cần tính lại thông tin, độ phức tạp thời gian là

$$O\left(B + \frac{B}{2} + \frac{B}{4} + \frac{B}{8} + \dots\right) = O(B).$$

Vấn đề thứ hai là khi hỏi đáp án của một khối, ta cần tìm kiếm nhị phân để biết đáp án nằm ở đoạn nào của hàm từng đoạn. Nếu tìm kiếm nhị phân trực tiếp, độ phức tạp của một thao tác là $O(B + n \log B/B)$, chưa đủ tốt. Cách trong lời giải chính thức là tối ưu tương tự mục 2.3, tức dùng sắp xếp cơ số rồi tận dụng tính đơn điệu. Độ phức tạp thời gian là $O(n\sqrt{n})$. Do xử lý từng khối, ta có thể chỉ xây dựng cây đoạn của khối đang được xử lý, nên độ phức tạp không gian là $O(B \log B + n) = O(n)$, chứ không phải $O(n \log n)$ như khi xây tất cả cây đoạn của mọi khối.

Thực ra ta có thể dùng phương pháp fractional cascading của thuật toán 4 ở mục 3.2, trong đó dãy được lưu ở các lá là dãy gồm các điểm phân chia của hàm từng đoạn. Với khối được phủ hoàn toàn, thao tác sửa chỉ tịnh tiến hàm từng đoạn, tức cộng cùng một số vào mọi điểm phân chia. Vì vậy nếu đoạn tương ứng với một đỉnh trong cây đoạn được phủ hoàn toàn, mọi dãy mà các đỉnh trong cây con của nó duy trì cũng chỉ bị cộng cùng một số vào mọi phần tử, không làm thay đổi cấu trúc. Do đó cách làm ở mục 3.2 vẫn áp dụng được, với độ phức tạp thời gian $O(n\sqrt{n})$ và độ phức tạp không gian $O(n \log n)$.

3.4 Kết luận

Fractional cascading tuy đã có hơn 20 năm lịch sử và là một thuật toán phổ biến, nhưng hiện vẫn chưa được ứng dụng rộng rãi trong thi tin học. Bài viết này giới thiệu fractional cascading, so sánh thuật toán này với các thuật toán khác, và áp dụng nó vào một số bài toán trong thi tin học, đặc biệt là tối ưu thuật toán khi giải dạng bài chia khối cổ điển này. Hy vọng bài viết có ích cho mọi người.

Tuy nhiên, chưa có nhiều người nghiên cứu lĩnh vực này, và do trình độ tác giả còn hạn chế, vẫn còn nhiều vấn đề chưa được giải quyết. Mong bài viết này có thể gợi mở thêm thảo luận và thu hút nhiều đồng nghiệp hơn nghiên cứu thuật toán này.

Lời cảm ơn

Xin cảm ơn China Computer Federation đã cung cấp nền tảng học tập và trao đổi. Xin cảm ơn huấn luyện viên Gao Wenyuan của đội tuyển quốc gia đã hướng dẫn. Xin cảm ơn các thầy Lin Yang, Zhang Junliang, Hu Fan, Li Zhiwu, và Lin Hong đã bồi dưỡng và chỉ dạy tôi. Xin cảm ơn các anh Cai Chengze, Li Xinlong, v.v. đã cho tôi cảm hứng trong các cuộc thảo luận. Xin cảm ơn các anh Wang Xiuhuan, Wang Siqi, Yuan Fangzhou, v.v. đã chỉ dẫn. Xin cảm ơn toàn thể các bạn lớp tin học khóa 2018 của Trường Trung học số 7 Thành Đô đã đồng hành trong hai năm. Xin cảm ơn cha mẹ đã thấu hiểu và ủng hộ tôi.

Tài liệu tham khảo

1. Chazelle, Bernard; Guibas, Leonidas J. *Fractional cascading: I. A data structuring technique*. 1986.
2. Chazelle, Bernard; Guibas, Leonidas J. *Fractional cascading: II. Applications*. 1986.
3. Wikipedia contributors. *Fractional cascading*. Wikipedia. https://en.wikipedia.org/wiki/Fractional_cascading.
4. Blog CSDN của Baimafujinji: blog.csdn.net/baimafujinji/.../52956605.

4 Bàn về cây thống trị và ứng dụng

Chen Sunli, Trường Ngoại ngữ Nam Kinh

Tóm tắt

Bài viết này chủ yếu giới thiệu thuật toán tìm cây thống trị và các ứng dụng của nó.

Mục 2 giới thiệu một số ký hiệu và định nghĩa. Mục 3 tập trung trình bày thuật toán tìm cây thống trị. Mục 4 giới thiệu một số ứng dụng của cây thống trị. Mục 5 là phần tổng kết.

4.1 Mở đầu

Khái niệm cây thống trị thường xuất hiện trong lý thuyết đồ thị và lý thuyết thiết kế máy tính; với tư cách một thuật toán, nó cũng thỉnh thoảng xuất hiện trong các kỳ thi thuật toán. Tuy nhiên, tài liệu tiếng Trung giới thiệu có hệ thống về nó còn tương đối ít. Tác giả hy vọng bài viết này có thể giúp nhiều bạn học sinh hiểu cây thống trị hơn, cũng như hiểu một vài ứng dụng của nó.

4.2 Định nghĩa

Trong bài viết này ta xét một đồ thị có hướng $G = (V, E)$ gồm n đỉnh và m cạnh, trong đó V là tập đỉnh, E là tập cạnh, và $|V| = n, |E| = m$. Bài viết giả sử người đọc đã quen với các khái niệm cơ bản trong lý thuyết đồ thị như đường đi, chu trình, tìm kiếm theo chiều sâu (DFS), thứ tự topo, thứ tự DFS, v.v.

Khi đã cho một nguồn $s \in V$, nếu với hai đỉnh x, y , mọi đường đi bắt đầu từ s và kết thúc ở y đều đi qua x , thì ta nói x thống trị (*dominates*) y . Vì khi không tồn tại đường đi từ s đến x , các quan hệ thống trị liên quan tới x đều không có ý nghĩa, nên trong bài viết này ta luôn giả sử từ s có thể tới được mọi đỉnh khác. Với đồ thị tổng quát, chỉ cần duyệt hết duyệt sâu từ s và chỉ giữ lại các đỉnh được thăm.

Trước hết quan sát rằng ta chỉ cần xét các đường đi đơn. Lý do là nếu một đường đi xuất hiện đỉnh lặp lại, ta có thể xóa các đỉnh giữa hai lần xuất hiện lặp đó; phép sửa này không ảnh hưởng tới điều kiện

thống trị. Một tính chất hiển nhiên là s thống trị mọi đỉnh. Ngoài ra, quan hệ thống trị còn thỏa các tính chất dưới đây, nhờ đó có thể dùng một cấu trúc cây để mô tả đầy đủ chúng.

Tính chất 1. Nếu x thống trị y và y thống trị z , thì x thống trị z . Nói cách khác, quan hệ thống trị có tính bắc cầu.

Tính chất này khá hiển nhiên.

Tính chất 2. Nếu x thống trị y và y thống trị x , thì $x = y$.

Chứng minh. Giả sử $x \neq y$. Khi đó mọi đường đi đơn từ s đến x đều đi qua y , và trước khi đi qua y lại bắt buộc phải đi qua x . Vì vậy đường đi có dạng $s, \dots, x, \dots, y, \dots, x$, mâu thuẫn với tính đơn. Suy ra bắt buộc có $x = y$. $\square$

Tính chất 3. Nếu x, y, z đôi một khác nhau, và cả x lẫn y đều thống trị z , thì hoặc x thống trị y , hoặc y thống trị x .

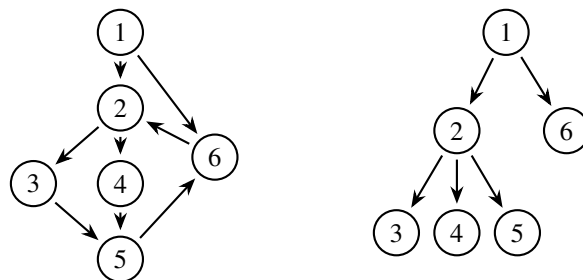
Chứng minh. Xét một đường đi đơn bất kỳ từ s đến z . Đường đi này chắc chắn chứa cả x lẫn y . Không mất tính tổng quát, giả sử đường đi có dạng $s, \dots, x, \dots, y, \dots, z$. Nếu x không thống trị y , thì tồn tại một đường đi từ s đến y không qua x . Khi đó thay phần từ s đến y của đường đi nói trên bằng đường đi không qua x này, ta thu được một đường đi từ s đến z không qua x , mâu thuẫn với việc x thống trị z . $\square$

Các tính chất trên cho thấy: với một đỉnh $x \neq s$, xét tập

$$S = \{y \mid y \text{ thống trị } x\}.$$

Các đỉnh trong S tạo thành một tập được sắp toàn phần theo quan hệ thống trị, và S có ít nhất hai phần tử. Do đó chắc chắn tồn tại một đỉnh z thỏa: nếu $y \neq x$ thống trị x , thì y cũng thống trị z . Ta gọi z là điểm thống trị trực tiếp (*immediate dominator*) của x , ký hiệu $z = \text{idom}(x)$.

Bây giờ, với mọi đỉnh x khác s , nối một cạnh $\text{idom}(x) \rightarrow x$. Đồ thị thu được có bậc vào của mỗi đỉnh không quá 1. Kết hợp Tính chất 1 và Tính chất 2, đồ thị này không có chu trình, vì thế nó chắc chắn là một cây có hướng gốc s , gọi là cây thống trị (*Dominator Tree*), ký hiệu $DT(G, s)$. Nó thỏa: với hai đỉnh x, y , x thống trị y khi và chỉ khi trên cây thống trị x là tổ tiên của y (trong bài viết này, tổ tiên bao gồm chính đỉnh đó). Vì vậy, chỉ cần tìm được cây thống trị, ta tìm được toàn bộ các quan hệ thống trị.



Hình 2: hình bên phải là cây thống trị của hình bên trái khi $s = 1$.

4.3 Thuật toán tìm cây thống trị

4.3.1 Trường hợp đặc biệt: đồ thị có hướng không chu trình

Với đồ thị có hướng không chu trình $G = (V, E)$, do các đỉnh đứng sau trong thứ tự topo không ảnh hưởng tới quan hệ thống trị của các đỉnh đứng trước, ta có thể lần lượt tìm điểm thống trị trực tiếp của

từng đỉnh x theo thứ tự topo. Giả sử điểm thống trị trực tiếp của mọi đỉnh đứng trước x trong thứ tự topo đã được tìm. Khi đó ta có một cây thống trị chưa hoàn chỉnh, chắc chắn là một phần của cây thống trị cuối cùng; tạm gọi nó là “cây thống trị hiện tại”. Giả sử các cạnh kết thúc ở x là $(c_1, x), \dots, (c_k, x)$, khi đó $\text{idom}(c_i)$ đều đã được tìm.

Bổ đề 1. Đỉnh y thống trị x khi và chỉ khi $y = x$, hoặc y thống trị tất cả $c_1, \dots, c_k$.

Chứng minh. Khi $y = x$, kết luận hiển nhiên. Sau đây giả sử $y \neq x$. Trước hết chứng minh nếu y thống trị tất cả $c_1, \dots, c_k$ thì y chắc chắn thống trị x : xét một đường đi bất kỳ từ s đến x , đỉnh ngay trước x chắc chắn là một trong $c_1, \dots, c_k$, nên đường đi này bắt buộc đi qua y .

Tiếp theo chứng minh mệnh đề phản đảo của chiều trên: không mất tính tổng quát, giả sử y không thống trị c_1 . Khi đó tồn tại đường đi từ s đến c_1 không qua y ; đi tiếp theo cạnh (c_1, x) , ta thu được một đường đi từ s đến x không qua y . Vì vậy y không thống trị x . $\square$

Các đỉnh thỏa tính chất trên chính là tổ tiên chung của $c_1, \dots, c_k$ trên “cây thống trị hiện tại”. Vì vậy

$$\text{idom}(x) = \text{LCA}(\text{idom}(c_1), \dots, \text{idom}(c_k)),$$

trong đó LCA là tổ tiên chung gần nhất. Điều này tương đương với việc thêm một lá và một cạnh vào “cây thống trị hiện tại”. Nếu dùng thuật toán LCA bằng nhảy bậc hai, chỉ cần sửa nhẹ là có thể hỗ trợ mỗi lần thêm một lá. Độ phức tạp thời gian và không gian của cách làm này đều là $O((n + m) \log n)$.

4.3.2 Định nghĩa và tính chất của điểm bán thống trị

Trong thuật toán sau đây, ta cần xét cây DFS của đồ thị tùy ý G xuất phát từ s , tức cấu trúc cây do các cạnh được đi qua trong quá trình duyệt sâu tạo thành; đồng thời gán thứ tự cho các đỉnh theo thứ tự duyệt (thứ tự DFS). Cụ thể, với hai đỉnh x, y , nếu x được thăm sớm hơn y trong quá trình duyệt, ta viết $x < y$; tương tự có thể định nghĩa $x > y$, giá trị lớn nhất và nhỏ nhất của một tập đỉnh, v.v. Cây DFS của đồ thị có hướng không tồn tại cạnh (x, y) thỏa $x < y$ và x không phải tổ tiên của y . Một phát biểu hữu ích hơn là:

Định lý 1. Nếu $x \leq y$, thì mọi đường đi từ x đến y bắt buộc đi qua một tổ tiên chung nào đó của x và y trên cây DFS.

Chứng minh. Nếu x là tổ tiên của y , kết luận hiển nhiên, vì vậy có thể coi x không phải tổ tiên của y . Giả sử tồn tại một đường đi

$$x = v_0, v_1, \dots, v_k = y$$

trên đó không có tổ tiên chung nào của x và y trong cây DFS. Gọi z là tổ tiên chung gần nhất của x và y trên cây DFS; trong các con của z , chắc chắn có đúng một đỉnh y' là tổ tiên của y .

Xét đường đi bất kỳ $x = v_0, v_1, \dots, v_k = y$. Đặt

$$p = \max\{i \mid v_i < y'\},$$

và giả sử v_p không phải tổ tiên của z . Khi đó v_p cũng chắc chắn không phải tổ tiên của v_{p+1} . Điều này suy ra từ tính chất của thứ tự DFS: nếu x là tổ tiên của y , thì x cũng là tổ tiên của mọi z thỏa $x \leq z \leq y$. Từ đó có cạnh (v_p, v_{p+1}) , $v_p < v_{p+1}$, và v_p không phải tổ tiên của v_{p+1} . Nhưng cạnh này không thể tồn tại trong cây DFS, nếu không khi duyệt tới v_p ta sẽ đi trực tiếp theo cạnh đó để thăm v_{p+1} . Mâu thuẫn này cho thấy v_p chắc chắn là tổ tiên của z . $\square$

Sau đây là định nghĩa điểm bán thống trị (*semi-dominator*).

Định nghĩa. Điểm bán thống trị của đỉnh x là

$$\text{sdom}(x) = \min\{y \mid \text{tồn tại đường đi } y = v_0, v_1, \dots, v_k = x \text{ sao cho với mọi } 1 \leq i \leq k-1, v_i > x\}.$$

Ta gọi sự tồn tại của đường đi trong định nghĩa này là “điều kiện điểm bán thống trị”. Việc tìm điểm bán thống trị là bước trung gian để tìm điểm thống trị trực tiếp. Dưới đây là một số tính chất quan trọng về điểm thống trị trực tiếp và điểm bán thống trị.

Bổ đề 2. $\text{idom}(x)$ và $\text{sdom}(x)$ đều là tổ tiên của x trên cây DFS và đều khác x . Hơn nữa, trên cây DFS, $\text{idom}(x)$ là tổ tiên của $\text{sdom}(x)$.

Chứng minh. Việc $\text{idom}(x)$ là tổ tiên của x trên cây DFS là hiển nhiên, vì trên cây DFS có một đường đi từ s đến x , và $\text{idom}(x)$ bắt buộc nằm trên đường đi này.

Do cha của x trên cây DFS đã thỏa yêu cầu của điểm bán thống trị, nên $\text{sdom}(x) < x$. Mặt khác, nếu $\text{sdom}(x)$ không phải tổ tiên của x , thì không khó thấy cha của $\text{sdom}(x)$ trên cây DFS cũng thỏa yêu cầu của điểm bán thống trị, và còn nhỏ hơn $\text{sdom}(x)$, mâu thuẫn với định nghĩa điểm bán thống trị.

Theo định nghĩa điểm bán thống trị, ta có thể đi theo cây DFS từ s tới $\text{sdom}(x)$, rồi đi theo đường đi trong định nghĩa tới x . Vì vậy các đỉnh trên đường từ $\text{sdom}(x)$ tới x trong cây DFS (không bao gồm $\text{sdom}(x)$) đều không thống trị x . Do đó $\text{idom}(x)$ chắc chắn là tổ tiên của $\text{sdom}(x)$. $\square$

Bổ đề 3. Cho v, w thỏa v là tổ tiên của w trên cây DFS. Khi đó hoặc v là tổ tiên của $\text{idom}(w)$, hoặc $\text{idom}(w)$ là tổ tiên của $\text{idom}(v)$.

Chứng minh. Nếu $v = w$, kết luận hiển nhiên, nên giả sử $v \neq w$. Chứng minh bằng phản chứng. Vì $v, w, \text{idom}(v), \text{idom}(w)$ đều là tổ tiên của w , nếu kết luận trong bổ đề không đúng, thì theo thứ tự độ sâu chúng lần lượt là

$$\text{idom}(v), \text{idom}(w), v, w$$

và đôi một khác nhau. Điều này có nghĩa là tồn tại một đường đi từ s đến v không qua $\text{idom}(w)$. Đi tiếp theo cây DFS tới w , ta thu được một đường đi từ s đến w không qua $\text{idom}(w)$, mâu thuẫn. $\square$

Với các tính chất trên, ta sẽ chỉ ra mối liên hệ chặt chẽ giữa điểm bán thống trị và điểm thống trị trực tiếp, từ đó có thể dùng cái trước để tính cái sau.

Định lý 2. Lấy một đỉnh $w \neq s$. Xét mọi đỉnh x trên đường từ $\text{sdom}(w)$ tới w trong cây DFS, không bao gồm $\text{sdom}(w)$. Nếu chúng đều thỏa $\text{sdom}(x) \geq \text{sdom}(w)$, thì

$$\text{idom}(w) = \text{sdom}(w).$$

Chứng minh. Theo Bổ đề 2, $\text{idom}(w)$ là tổ tiên của $\text{sdom}(w)$, nên chỉ cần chứng minh $\text{sdom}(w)$ thống trị w . Xét một đường đi P bất kỳ từ s đến w ; ta sẽ chứng minh $\text{sdom}(w)$ chắc chắn nằm trên P . Gọi x là đỉnh cuối cùng trên P thỏa $x < \text{sdom}(w)$. Nếu không tồn tại đỉnh như vậy, thì chắc chắn $\text{sdom}(w) = \text{idom}(w) = s$, kết luận vẫn đúng. Đồng thời, gọi y là đỉnh đầu tiên sau x trên P nằm trên đường từ $\text{sdom}(w)$ tới w trong cây DFS. Lấy đoạn con từ x tới y trên P , được $v_0 = x, v_1, \dots, v_k = y$.

Bây giờ xét đường $v_0, \dots, v_k$. Ta khẳng định với mọi $1 \leq i < k$ đều có $v_i > y$, tức $\text{sdom}(y) \leq x < \text{sdom}(x)$. Nếu không, tồn tại một $v_i < y$. Khi đó theo Định lý 1, tồn tại $i \leq j \leq k-1$ sao cho v_j là tổ tiên của y . Theo cách chọn x , có $\text{sdom}(w) \leq v_j$, vì vậy v_j cũng nằm trên đường từ $\text{sdom}(w)$ tới w trong cây DFS, mâu thuẫn với cách chọn y . Mâu thuẫn trên suy ra $\text{sdom}(y) \leq x < \text{sdom}(x)$. Kết hợp với điều kiện của định lý, bắt buộc có $y = \text{sdom}(w)$, tức đường đi P chứa $\text{sdom}(w)$. $\square$

Định lý 3. Lấy một đỉnh $w \neq s$. Xét mọi đỉnh trên đường từ $\text{sdom}(w)$ tới w trong cây DFS, không bao gồm $\text{sdom}(w)$. Gọi u là đỉnh có giá trị sdom nhỏ nhất trong số đó. Khi đó $\text{sdom}(u) \leq \text{sdom}(w)$ và

$$\text{idom}(u) = \text{idom}(w).$$

Chứng minh. Vì w cũng là một ứng viên của u , nên $\text{sdom}(u) \leq \text{sdom}(w)$ là hiển nhiên. Chú ý rằng trên cây DFS, $\text{idom}(w)$ là tổ tiên của u . Theo Bổ đề 3, $\text{idom}(w)$ cũng là tổ tiên của $\text{idom}(u)$, nên chỉ cần chứng minh $\text{idom}(u)$ thống trị w .

Xét một đường đi P bất kỳ từ s đến w ; ta sẽ chứng minh $\text{idom}(u)$ chắc chắn nằm trên P . Gọi x là đỉnh cuối cùng trên P thỏa $x < \text{idom}(u)$. Nếu không tồn tại đỉnh như vậy, thì chắc chắn $\text{idom}(u) = \text{idom}(w) = s$, kết luận vẫn đúng. Đồng thời, gọi y là đỉnh đầu tiên sau x trên P nằm trên đường từ $\text{idom}(u)$ tới w trong cây DFS. Lấy đoạn con từ x tới y trên P , được $v_0 = x, v_1, \dots, v_k = y$.

Tương tự chứng minh Định lý 2, ta có $\text{sdom}(y) \leq x$. Theo Bổ đề 2, $\text{idom}(u) \leq \text{sdom}(u)$. Kết hợp lại:

$$\text{sdom}(y) \leq x < \text{idom}(u) \leq \text{sdom}(u).$$

Đến đây, từ cách chọn u , ta biết y không thể là hậu duệ thật sự của $\text{sdom}(w)$. Mặt khác, y cũng không thể vừa là hậu duệ thật sự của $\text{idom}(u)$, vừa là tổ tiên của u ; nếu không, đi theo cây DFS từ s tới $\text{sdom}(y)$, rồi đi theo P tới y , cuối cùng đi theo cây DFS tới u , ta thu được một đường đi không qua $\text{idom}(u)$, mâu thuẫn với định nghĩa điểm thống trị.

Theo $\text{idom}(u) \leq \text{sdom}(w) < u \leq w$ và $\text{idom}(u) < y$, khả năng duy nhất còn lại là $y = \text{idom}(u)$. Vậy đường đi P chứa $\text{idom}(u)$. $\square$

Hai định lý trên giải thích quan hệ giữa điểm bán thống trị và điểm thống trị trực tiếp, đồng thời cho một cách tính điểm thống trị trực tiếp từ điểm bán thống trị. Với đỉnh $w \neq s$, xét mọi đỉnh trên đường từ $\text{sdom}(w)$ tới w trong cây DFS, không bao gồm $\text{sdom}(w)$. Gọi u là đỉnh có sdom nhỏ nhất trong số đó. Nếu $\text{sdom}(u) = \text{sdom}(w)$, dùng Định lý 2 ta có $\text{idom}(w) = \text{sdom}(w)$; ngược lại, $u \neq w$, dùng Định lý 3 ta có $\text{idom}(w) = \text{idom}(u)$.

4.3.3 Tính điểm bán thống trị và điểm thống trị trực tiếp

Điểm bán thống trị. Để tìm điểm bán thống trị, ta cần định lý dưới đây.

Định lý 4. $\text{sdom}(w)$ bằng giá trị nhỏ nhất trong các ứng viên sinh ra từ hai trường hợp sau:

1. Có cạnh (v, w) ; khi đó v là một ứng viên của $\text{sdom}(w)$.
2. u là tổ tiên của v trên cây DFS, $u > w$, và có cạnh (v, w) ; khi đó $\text{sdom}(u)$ là một ứng viên của $\text{sdom}(w)$.

Chứng minh. Gọi x là giá trị nhỏ nhất trong mọi ứng viên. Trước hết ta chứng minh mỗi ứng viên đều thỏa điều kiện điểm bán thống trị, từ đó suy ra $\text{sdom}(w) \leq x$. Nếu x đến từ trường hợp 1, rõ ràng (x, w) chính là đường đi thỏa điều kiện điểm bán thống trị. Ngược lại, $x = \text{sdom}(u)$. Khi đó lấy đường đi từ $\text{sdom}(u)$ đến u được nêu trong điều kiện điểm bán thống trị, nối tiếp đường đi từ u đến v trong cây DFS, rồi cuối cùng đi từ v đến w , ta được một đường đi thỏa điều kiện điểm bán thống trị của w .

Còn cần chứng minh $\text{sdom}(w) \geq x$. Xét đường đi ứng với điều kiện điểm bán thống trị từ $\text{sdom}(w)$ đến w :

$$v_0 = \text{sdom}(w), v_1, \dots, v_k = w,$$

trong đó với mọi $1 \leq i < k$ đều có $v_i \geq w$. Nếu $k = 1$, nó đã được xét trong trường hợp 1; sau đây giả sử $k > 1$. Lấy j là chỉ số nhỏ nhất thỏa $j \geq 1$ và v_j là tổ tiên của v_{k-1} trên cây DFS. Chỉ số như vậy chắc chắn tồn tại, vì k là một chỉ số hợp lệ.

Bây giờ xét đường $v_0, \dots, v_j$. Ta khẳng định nó là đường thỏa điều kiện điểm bán thống trị của v_j , tức với mọi $1 \leq i < j$ đều có $v_i > v_j$. Nếu không, xét mọi chỉ số i thỏa $v_i \leq v_j$, lấy chỉ số có v_i nhỏ nhất. Theo Định lý 1, v_i chắc chắn là tổ tiên của v_j , mâu thuẫn với cách chọn j . Vì vậy $\text{sdom}(v_j) \leq \text{sdom}(w)$; mà $\text{sdom}(v_j)$ chắc chắn được xét trong trường hợp 2, nên $x \leq \text{sdom}(w)$. $\square$

Với Định lý 4, có thể thiết kế thuật toán tìm điểm bán thống trị như sau:

1. Tìm một cây DFS của G và thứ tự DFS.
2. Nếu điểm sdom của mọi đỉnh đều đã được tìm, kết thúc thuật toán; ngược lại, tìm đỉnh lớn nhất w trong các đỉnh chưa có $\text{sdom}(w)$.
3. Xét trường hợp 1, có thể tính trực tiếp mọi ứng viên.
4. Xét trường hợp 2: duyệt các cạnh (v, w) thỏa $v > w$. Khi đó v chắc chắn đã được đánh dấu. Bắt đầu từ v , liên tục đi lên cha đã được đánh dấu trên cây DFS để tạo thành một đường đi, rồi lấy giá trị sdom nhỏ nhất trên đường này làm ứng viên.
5. Gộp các ứng viên thu được từ trường hợp 1 và 2 để nhận giá trị $\text{sdom}(w)$, đánh dấu w là đã tìm được sdom , rồi quay lại bước 2.

Có thể thấy việc xét mọi đỉnh theo thứ tự từ lớn xuống nhỏ chính là để các giá trị sdom cần dùng trong bước 4 đều đã được tìm. Để thấy các phần ngoài bước 4 có tổng độ phức tạp không vượt quá $O(n + m)$. Để tối ưu độ phức tạp, ở bước 4 ta dùng cấu trúc dữ liệu DSU có trọng số để duy trì giá trị sdom nhỏ nhất trên đường đi và vị trí đạt giá trị nhỏ nhất đó. Khi đánh dấu x , nối mọi con của x trên cây DFS tới x và duy trì lại thông tin nhỏ nhất. Nếu cài đặt DSU với nén đường đi một cách mộc mạc, độ phức tạp thời gian đạt $O((n + m) \log n)$, độ phức tạp không gian là $O(n + m)$. Với bài toán này còn có các cách cài đặt DSU nhanh hơn, đạt độ phức tạp thấp hơn là $O((n + m)\alpha(n))$, thậm chí tuyến tính; bạn đọc quan tâm có thể tham khảo tài liệu liên quan.

Điểm thống trị trực tiếp. Để tìm mọi idom, định nghĩa $\text{pivot}(w)$ là đỉnh có giá trị sdom nhỏ nhất trên đường từ $\text{sdom}(w)$ tới w trong cây DFS, không bao gồm $\text{sdom}(w)$. Nếu đã tìm được mọi $\text{sdom}(w)$ và $\text{pivot}(w)$, ta có thể lần lượt xác định idom của từng đỉnh theo thứ tự từ nhỏ tới lớn dựa trên Bổ đề 2 và Bổ đề 3. Cụ thể: nếu

$$\text{sdom}(\text{pivot}(w)) = \text{sdom}(w),$$

thì $\text{idom}(w) = \text{sdom}(w)$; ngược lại

$$\text{idom}(w) = \text{idom}(\text{pivot}(w)).$$

Ở đây, do $\text{pivot}(w) \leq w$, trước khi xét tới w thì $\text{idom}(\text{pivot}(w))$ chắc chắn đã được tìm.

Cuối cùng, chỉ cần sửa nhẹ thuật toán tìm điểm bán thống trị ở trên là có thể tìm được pivot của mỗi đỉnh. Với mỗi đỉnh x , mở một $\text{bucket}(x)$ để lưu mọi đỉnh w thỏa $\text{sdom}(w) = x$. Nếu hiện đang tìm sdom của đỉnh w , thì trước khi thực hiện thao tác đánh dấu ở bước 5, tức thao tác *link* của DSU, xét mọi đỉnh x trong $\text{bucket}(w)$ và truy vấn trực tiếp trong DSU để lấy giá trị $\text{pivot}(x)$. Sau khi tìm được $\text{sdom}(w)$, cũng đừng quên thêm w vào $\text{bucket}(\text{sdom}(w))$.

Kết hợp những điều trên, ta đã có thuật toán cây thống trị hoàn chỉnh.

4.3.4 Tổng kết thuật toán

Dưới đây là phần tổng hợp và tóm lược thuật toán:

1. Tìm một cây DFS của G và thứ tự DFS.
2. Nếu điểm $sdom$ của mọi đỉnh đều đã được tìm, chuyển tới bước 8; ngược lại, tìm đỉnh lớn nhất w trong các đỉnh chưa có $sdom(w)$.
3. Xét trường hợp 1, có thể tính trực tiếp mọi ứng viên.
4. Xét trường hợp 2: duyệt các cạnh (v, w) thỏa $v > w$. Khi đó v chắc chắn đã được đánh dấu. Bắt đầu từ v , liên tục đi lên cha đã được đánh dấu trên cây DFS để tạo thành một đường đi, rồi lấy giá trị $sdom$ nhỏ nhất trên đường này làm ứng viên. Đường này chính là đường từ đỉnh v tới gốc của v trong DSU hiện tại, nên có thể truy vấn trực tiếp trong DSU.
5. Gộp các ứng viên thu được từ trường hợp 1 và 2 để nhận giá trị $sdom(w)$, rồi thêm w vào $bucket(sdom(w))$.
6. Với mỗi đỉnh x trong $bucket(w)$, tìm $pivot(x)$. Giá trị này cũng có thể lấy trực tiếp bằng cách truy vấn giá trị nhỏ nhất trên đường từ x tới gốc của x trong DSU.
7. Với mỗi con c của w trên cây DFS, đặt cha của c trong DSU là w , đồng thời duy trì thông tin giá trị nhỏ nhất trên đường.
8. Xét lần lượt từng đỉnh w theo thứ tự từ nhỏ tới lớn. Nếu $sdom(pivot(w)) = sdom(w)$, đặt $idom(w) = sdom(w)$; ngược lại đặt $idom(w) = idom(pivot(w))$.

Thuật toán này có độ phức tạp không gian $O(n + m)$. Nút thắt thời gian nằm ở việc duy trì DSU; nếu dùng nén đường đi mộc mạc, độ phức tạp là $O((n + m) \log n)$. Trong phạm vi thi tin học, độ phức tạp này đã đủ tốt.

4.4 Ứng dụng của cây thống trị

4.4.1 Điểm thống trị của một tập đỉnh

Điểm thống trị của tập đỉnh S được định nghĩa là đỉnh mà mọi đường đi từ s tới bất kỳ $x \in S$ nào cũng bắt buộc đi qua. Không khó thấy các đỉnh như vậy chính là giao của các tập điểm thống trị của mọi đỉnh trong S . Mà điểm thống trị của đỉnh x chính là mọi tổ tiên của x trên cây thống trị, nên điểm thống trị của tập S chính là tổ tiên chung gần nhất (LCA) của mọi $x \in S$ trên cây thống trị.

4.4.2 Cạnh thống trị

Tương tự định nghĩa điểm thống trị, nếu mọi đường đi từ s tới x đều đi qua cạnh (u, v) , thì ta nói (u, v) thống trị x . Đồng thời cũng có thể định nghĩa cạnh thống trị trực tiếp một cách tương tự: nếu (u, v) thống trị x , và mọi cạnh thống trị của x cũng thống trị u , thì gọi (u, v) là cạnh thống trị trực tiếp của x .

Để tìm cạnh thống trị của mỗi đỉnh, một cách đơn giản là với mỗi cạnh (u, v) trong đồ thị gốc, tạo một đỉnh phụ w , nối (u, w) và (w, v) , rồi tìm cây thống trị của đồ thị mới. Với một đỉnh x của đồ thị gốc, các cạnh thống trị của nó chính là các tổ tiên là đỉnh phụ của x trong cây thống trị của đồ thị mới; cạnh thống trị trực tiếp là đỉnh có độ sâu lớn nhất trong số đó, có thể tìm đơn giản bằng DFS. Tuy nhiên, đồ thị mới có $n + m$ đỉnh và $2m$ cạnh, có thể gây hăng số khá lớn. Xem kỹ cách làm này, ta thấy các đỉnh phụ có nhiều tính chất trong quá trình chạy thuật toán, nhờ đó không nhất thiết phải chạy trọn vẹn thuật toán trên đồ thị mới mới suy ra được cạnh thống trị.

Bổ đề 4. Cạnh (u, v) là cạnh thống trị của đỉnh x khi và chỉ khi u và v đều là điểm thống trị của x , đồng thời chỉ có đúng một đường đi đơn từ u tới v , tức cạnh (u, v) là cạnh thống trị của v .

Chứng minh. Nếu cạnh (u, v) là cạnh thống trị của x , thì mọi đường đi từ s tới x đều đi qua u và v , tức u và v thống trị x . Hơn nữa, nếu từ u tới v còn một đường đi khác ngoài cạnh (u, v) , ta có thể đi theo một đường đi bất kỳ từ s tới u , rồi tới v , rồi tới x mà không qua (u, v) . Vì vậy từ u tới v bắt buộc có đúng một đường đi đơn.

Ngược lại, nếu các điều kiện của bổ đề đúng, thì hiển nhiên cạnh (u, v) sẽ thống trị đỉnh x . $\square$

Từ Bổ đề 4, với một cạnh (u, v) trên cây DFS, nếu u thống trị v và từ u tới v chỉ có đúng một đường đi đơn, thì (u, v) thống trị mọi đỉnh mà v thống trị. Nhìn lại thuật toán cây thống trị, nếu cạnh (u, v) nằm trên cây DFS và $\text{idom}(v) = u$, thì chắc chắn có $\text{sdom}(v) = u$.

Bây giờ tập trung vào điều kiện “từ u tới v chỉ có đúng một đường đi đơn”. Có thể hiểu nó qua cách tạo đỉnh phụ ở trên. Giả sử chỉ tạo đỉnh phụ w cho riêng cạnh này, khi đó $\text{sdom}(w) = u$ và $\text{sdom}(v) \geq u$. Lúc này $\text{idom}(v) = w$ khi và chỉ khi $\text{sdom}(v) = w$. Quay lại đồ thị gốc, điều này tương đương với: khi tính $\text{sdom}(v)$, không xét cạnh (u, v) , tức khi chỉ xét trường hợp 2 thì không có ứng viên nào. Như vậy, chỉ cần thêm một kiểm tra trong quá trình tìm sdom , ta có thể xác định liệu từ u tới v có chỉ một đường đi đơn hay không.

Cuối cùng, ta tìm được mọi cạnh (u, v) thỏa nó thống trị v , rồi duyệt một lần trên cây DFS là xác định được cạnh thống trị trực tiếp của mỗi đỉnh. Cách làm này không cần tạo đỉnh phụ, hằng số nhỏ hơn và độ khó cài đặt không tăng rõ rệt.

4.4.3 Điểm khớp và cầu trong đồ thị có hướng

Trong đồ thị có hướng G , u và v liên thông mạnh nếu u tới được v và v cũng tới được u . Quan hệ liên thông mạnh là quan hệ tương đương; các lớp tương đương mà nó tạo ra được gọi là thành phần liên thông mạnh, tương ứng với thành phần liên thông trong đồ thị vô hướng. Với đồ thị G , ký hiệu $G \setminus \{u\}$ là đồ thị mới thu được sau khi xóa đỉnh u và các cạnh liên quan; $G \setminus \{(u, v)\}$ là đồ thị mới thu được sau khi xóa cạnh (u, v) .

Điểm khớp và cầu của đồ thị vô hướng là các đỉnh và cạnh mà sau khi xóa, số thành phần liên thông tăng lên. Tương ứng với chúng, điểm khớp mạnh và cầu mạnh trong đồ thị có hướng là các đỉnh và cạnh mà sau khi xóa, số thành phần liên thông mạnh tăng lên. Một câu hỏi tự nhiên là làm thế nào để tìm tất cả điểm khớp mạnh và cầu mạnh trong một đồ thị G . Giống như đồ thị vô hướng, khi tìm điểm khớp mạnh và cầu mạnh, có thể xét riêng từng thành phần liên thông mạnh, nên phần thảo luận dưới đây đều dành cho đồ thị liên thông mạnh.

Bổ đề 5.1. Đỉnh v là điểm khớp mạnh khi và chỉ khi với mọi $x \neq v$, đều tồn tại một đỉnh $y \neq v$ sao cho một trong hai điều sau đúng:

1. Mọi đường đi từ x tới y đều đi qua v .
2. Mọi đường đi từ y tới x đều đi qua v .

Bổ đề 5.2. Cạnh (u, v) là cầu mạnh khi và chỉ khi với mọi x , đều tồn tại một đỉnh y sao cho một trong hai điều sau đúng:

1. Mọi đường đi từ x tới y đều đi qua cạnh (u, v) .
2. Mọi đường đi từ y tới x đều đi qua cạnh (u, v) .

Chứng minh. Chứng minh của hai bổ đề gần như giống nhau; ở đây chỉ chứng minh Bổ đề 5.1. Nếu v là điểm khớp mạnh, thì $G \setminus \{v\}$ không phải đồ thị liên thông mạnh. Khi đó xét một đỉnh y không cùng thành phần liên thông mạnh với x : hoặc không tồn tại đường đi từ x tới y , hoặc không tồn tại đường đi từ y tới x . Do G là liên thông mạnh, suy ra điều kiện của bổ đề đúng. Mặt khác, nếu điều kiện của bổ đề đúng, thì x và y không còn liên thông mạnh sau khi xóa v . $\square$

Nếu dùng Bổ đề 5.1, chọn tùy ý một đỉnh v là có thể tìm mọi điểm khớp mạnh khác v . Với điều kiện 1, chỉ cần tìm $DT(G, v)$; mọi đỉnh không phải lá, trừ v , trong cây thống trị đều là điểm khớp mạnh. Với điều kiện 2, chỉ cần xây đồ thị đảo G^R của G rồi tìm $DT(G^R, v)$. Cuối cùng, dùng thuật toán tìm thành phần liên thông mạnh thông thường là có thể xác định liệu đỉnh v có phải điểm khớp mạnh hay không.

Với cầu mạnh cũng có kết luận tương tự, có thể áp dụng thuật toán cạnh thống trị ở mục trước để giải quyết. Các thuật toán tìm điểm khớp mạnh và cầu mạnh nói trên có độ phức tạp thời gian bằng với độ phức tạp tìm cây thống trị.

4.4.4 Ứng dụng trong công nghiệp

Cây thống trị có nhiều công dụng trong các cấu trúc cần dựa trên đồ thị có hướng.

Mạng nơ-ron có thể được trừu tượng hóa thành một đồ thị có hướng: nhiệm vụ tính toán ở mỗi đỉnh phụ thuộc vào kết quả của các đỉnh có cạnh trỏ tới nó. Để tăng tốc việc chạy mạng nơ-ron, người ta thường dùng tính toán song song để tối ưu thứ tự tính các đỉnh. Khi đó cây thống trị có thể giúp tìm một phần các đỉnh được sử dụng với tần suất cao và ưu tiên tính chúng.

Ngoài ra, cây thống trị còn có thể phát huy tác dụng trong nguyên lý biên dịch và phát hiện rò rỉ bộ nhớ. Nhìn chung, khái niệm “thống trị” có các cách nói tương tự trong nhiều lĩnh vực, nên cây thống trị, với tư cách công cụ tìm quan hệ thống trị, có phạm vi ứng dụng rộng.

4.5 Tổng kết

Bài viết này giới thiệu thuật toán tìm cây thống trị và một số ứng dụng. Dù bài toán cây thống trị đã được giải quyết, từ góc độ ứng dụng có thể thấy cây thống trị có nhiều biến thể và vẫn còn nhiều điều đáng nghiên cứu.

Hy vọng bài viết này giúp nhiều bạn học sinh hiểu thuật toán và công cụ cây thống trị hơn, đồng thời đi sâu khám phá các vấn đề liên quan.

Lời cảm ơn

Xin cảm ơn China Computer Federation đã cung cấp nền tảng học tập và trao đổi. Xin cảm ơn huấn luyện viên Gao Wenyuan của đội tuyển quốc gia đã hướng dẫn. Xin cảm ơn thầy Li Shu của Trường Ngoại ngữ Nam Kinh đã hướng dẫn và ủng hộ. Xin cảm ơn tất cả thầy cô và bạn học từng giúp đỡ tôi. Xin cảm ơn cha mẹ đã luôn hết lòng ủng hộ và khích lệ tôi.

Tài liệu tham khảo

1. Thomas Lengauer, Robert Endre Tarjan. *A Fast Algorithm for Finding Dominators in a Flowgraph*.
2. Adam L. Buchsbaum, Haim Kaplan, Anne Rogers, Jeffery R. Westbrook. *A New, Simpler Linear-Time Dominators Algorithm*.
3. Giuseppe F. Italiano, Luigi Laura, Federico Santaroni. *Finding Strong Bridges and Strong Articulation Points in Linear Time*.

4. Robert E. Tarjan. *Depth-First Search and Linear Graph Algorithms*.

5 Báo cáo ra đề *Ghép số*

Jiang Xunchi, Trường Trung học Xuejun Hàng Châu

Tóm tắt

Bài viết này giới thiệu bài *Ghép số* do tác giả ra cho một buổi luyện tập nội bộ của trường. Quá trình giải bắt đầu từ một thuật toán đơn giản, sau đó liên tục quan sát tính chất và tối ưu độ phức tạp, cuối cùng thu được một cách giải khéo léo.

5.1 Mô tả bài toán

5.1.1 Ý chính bài toán

Định nghĩa $f(i)$ là xâu thu được khi viết dạng nhị phân của i ; chẳng hạn $f(3) = "11"$, $f(6) = "110"$.

Cần chọn một hoán vị P của các số từ 1 tới n sao cho xâu

$$S = f(P_1) + f(P_2) + \dots + f(P_n)$$

có thứ tự từ điển nhỏ nhất có thể.

Hãy xuất số lượng ký tự 1 trong k vị trí đầu của S .

Vì một vài lý do, dữ liệu vào và ra đều dùng dạng nhị phân.

5.1.2 Quy mô dữ liệu

Với mọi dữ liệu, $1 \leq n < 2^{2000}$, $1 \leq k \leq |S|$.

- Subtask 1 (7 điểm): $1 \leq n \leq 8$;
- Subtask 2 (16 điểm): $1 \leq n \leq 10^5$;
- Subtask 3 (26 điểm): $1 \leq n \leq 2^{60}$;
- Subtask 4 (23 điểm): $1 \leq n \leq 2^{300}$;
- Subtask 5 (28 điểm): không có ràng buộc đặc biệt.

Giới hạn thời gian: 1s.

Giới hạn bộ nhớ: 512MB.

5.1.3 Định dạng đầu vào

Đọc dữ liệu từ đầu vào chuẩn.

Dòng đầu tiên là một xâu 01 không có số 0 đứng đầu, cho n trong dạng nhị phân.

Dòng thứ hai là một xâu 01 không có số 0 đứng đầu, cho k trong dạng nhị phân.

5.1.4 Định dạng đầu ra

Ghi ra đầu ra chuẩn.

Gồm một dòng, là một xâu 01 không có số 0 đứng đầu, cho đáp án của bài toán trong dạng nhị phân.

5.1.5 Ví dụ đầu vào

1011

10010

5.1.6 Ví dụ đầu ra

1000

5.1.7 Giải thích ví dụ

Theo đề bài, một hoán vị có thể là (mọi số dưới đây đều được viết trong dạng nhị phân):

1000, 100, 1001, 10, 1010, 101, 1011, 110, 1, 11, 111.

Ghép các biểu diễn nhị phân này nối tiếp nhau thành xâu S , để thấy đáp án là 8.

5.2 Ký hiệu và quy ước

Với một xâu S , định nghĩa $S[i]$ là ký tự thứ i của S .

Định nghĩa $S[l \dots r]$ là xâu tạo bởi các ký tự từ vị trí thứ l tới vị trí thứ r của S . Đặc biệt, nếu $l > r$, thì $S[l \dots r] = \emptyset$.

Định nghĩa $|S|$ là độ dài của S .

Định nghĩa $h(S)$ là số nhị phân thu được khi viết xâu 01 S từ trái sang phải.

Định nghĩa $|f(n)| = N$.

Định nghĩa $x \times S$ là xâu thu được sau khi lặp xâu S đúng x lần.

Định nghĩa S^∞ là xâu vô hạn thu được bằng cách lặp vô hạn xâu S .

Định nghĩa thứ tự từ điển như sau: nếu $|X| < |Y|$ và $X = Y[1 \dots |X|]$, hoặc tồn tại một $i \leq |X|$ thỏa

$$X[1 \dots i - 1] = Y[1 \dots i - 1] \quad \text{và} \quad X[i] < Y[i],$$

thì thứ tự từ điển của X nhỏ hơn Y .

5.3 Phân tích ban đầu

5.3.1 Thuật toán một

Liệt kê mọi hoán vị hợp lệ P , rồi tìm xâu có thứ tự từ điển nhỏ nhất trong các xâu được tạo ra.

Độ phức tạp là $O(n! \times n \log n)$.

Điểm kỳ vọng: 7 điểm.

5.3.2 Thuật toán hai

Bổ đề 1. Với hai xâu X và Y , nếu $X^\infty = Y^\infty$, thì $X + Y = Y + X$.

Chứng minh. Tìm chu kỳ nhỏ nhất s của X^∞ , khi đó có

$$X = \frac{|X|}{|s|} \times s, \quad Y = \frac{|Y|}{|s|} \times s.$$

Do đó

$$X + Y = Y + X = \frac{|X| + |Y|}{|s|} \times s.$$

□

Bổ đề 2. Với hai xâu X và Y , nếu chọn một $1 \leq k \leq |X| + |Y|$ thỏa

$$X^\infty[1 \dots k - 1] = Y^\infty[1 \dots k - 1],$$

thì

$$(X + Y)[k] = X^\infty[k],$$

$$(Y + X)[k] = Y^\infty[k].$$

Chứng minh. Không mất tính tổng quát, giả sử $|X| < |Y|$.

Nếu $k \leq |X|$, thì

$$(X + Y)[k] = X[k] = (X^\infty)[k],$$

$$(Y + X)[k] = Y[k] = (Y^\infty)[k].$$

Nếu $|X| < k \leq |Y|$, thì

$$(X + Y)[k] = Y[k - |X|] = (X^\infty)[k - |X|] = (X^\infty)[k],$$

$$(Y + X)[k] = Y[k] = (Y^\infty)[k].$$

Nếu $|Y| < k$, thì

$$(X + Y)[k] = Y[k - |X|] = (X^\infty)[k - |X|] = (X^\infty)[k],$$

$$(Y + X)[k] = X[k - |Y|] = (Y^\infty)[k - |Y|] = (Y^\infty)[k].$$

□

Bổ đề 3. Với hai xâu X và Y , nếu $X^\infty < Y^\infty$, thì

$$X + Y < Y + X.$$

Chứng minh. Giả sử vị trí khác nhau đầu tiên của X^∞ và Y^∞ là k . Theo Bổ đề 2, ta có

$$(X + Y)[1 \dots k] = (X^\infty)[1 \dots k],$$

$$(Y + X)[1 \dots k] = (Y^\infty)[1 \dots k].$$

Suy ra

$$(X + Y)[1 \dots k] < (Y + X)[1 \dots k],$$

tức $X + Y < Y + X$. □

Định lý 1. Cho n xâu $A_1, A_2, \dots, A_n$. Mọi hoán vị P làm cho

$$A_{P_1} + A_{P_2} + \dots + A_{P_n}$$

có thứ tự từ điển nhỏ nhất đều thỏa điều kiện sau: với mọi $1 \leq i < n$,

$$(A_{P_i})^\infty \leq (A_{P_{i+1}})^\infty.$$

Chứng minh. Nếu tồn tại một i thỏa

$$(A_{P_i})^\infty > (A_{P_{i+1}})^\infty,$$

thì theo Bổ đề 3,

$$A_{P_i} + A_{P_{i+1}} > A_{P_{i+1}} + A_{P_i}.$$

Vậy đổi chỗ P_i và P_{i+1} sẽ tốt hơn.

Còn với một cặp thỏa

$$(A_{P_i})^\infty = (A_{P_{i+1}})^\infty,$$

theo Bổ đề 1 ta có

$$A_{P_i} + A_{P_{i+1}} = A_{P_{i+1}} + A_{P_i}.$$

Do đó đổi chỗ hai xâu có $(A_{P_i})^\infty$ và $(A_{P_{i+1}})^\infty$ bằng nhau không ảnh hưởng tới xâu thu được; tức các xâu có giá trị A^∞ bằng nhau có thể được xếp theo thứ tự bất kỳ mà vẫn tối ưu. $\square$

Với $f(1), f(2), \dots, f(n)$, chỉ cần sắp xếp chúng theo thứ tự tăng dần của $f(i)^\infty$, rồi ghép lại, là thu được xâu S .

Độ phức tạp là $O(n \log^2 n)$.

Điểm kỳ vọng: 23 điểm.

5.4 Phân tích lời giải

Sắp xếp mọi số từ 1 tới n theo thứ tự tăng dần của $f(i)^\infty$; khi $f(i)^\infty$ bằng nhau thì sắp theo i tăng dần. Như vậy có thể xác định duy nhất một hoán vị P .

Định nghĩa $Q = P^{-1}$, tức $Q_{P_i} = i$.

Định nghĩa c là số nguyên duy nhất thỏa

$$\sum_{i=1}^{Q_c-1} |f(P_i)| \leq k,$$

$$\sum_{i=1}^{Q_c} |f(P_i)| > k.$$

Dễ thấy $f(c)$ chỉ giữ lại một tiền tố trong $S[1 \dots k]$, còn $f(P_1), f(P_2), \dots, f(P_{Q_c-1})$ xuất hiện đầy đủ trong $S[1 \dots k]$.

Lời giải tham khảo được chia thành hai phần. Phần thứ nhất là tìm giá trị c ; phần thứ hai là từ giá trị c tính ra đáp án. Vì phần thứ hai đơn giản hơn, dưới đây sẽ giới thiệu phần thứ hai trước.

5.5 Phần thứ hai

5.5.1 Tiền xử lý

Bổ đề 4. Nếu $i < j$ và $|f(i)| = |f(j)|$, thì $Q_i < Q_j$.

Chứng minh. Nếu $i < j$, $|f(i)| = |f(j)|$, thì $f(i)^\infty < f(j)^\infty$, tức vị trí của i đứng trước vị trí của j . Suy ra $Q_i < Q_j$. $\square$

Từ Bổ đề 4, mọi số có cùng độ dài đều xuất hiện trong P theo thứ tự tăng dần.

Định nghĩa

$$g(i, j) = \max\{k \mid |f(k)| = i, Q_k < Q_j\},$$

tức số có độ dài i đứng muộn nhất trong các số đứng trước j . Sau đây sẽ trình bày thuật toán tìm

$$g(1, c), g(2, c), \dots, g(N, c).$$

Định lý 2. Với hai xâu X và Y , nếu $X^\infty \neq Y^\infty$, gọi k là vị trí khác nhau đầu tiên của X^∞ và Y^∞ , thì

$$k \leq 2 \max(|X|, |Y|).$$

Chứng minh. Dưới đây sẽ chứng minh rằng nếu $2 \max(|X|, |Y|)$ ký tự đầu của X^∞ và Y^∞ bằng nhau, thì $X^\infty = Y^\infty$.

Nếu $\gcd(|X|, |Y|) = 1$, xét $|Y|$ ký tự đầu của X^∞ và Y^∞ , ta có

$$X[(i-1) \bmod |X| + 1] = Y[i].$$

Xét từ vị trí $|Y| + 1$ tới $2|Y|$ của X^∞ và Y^∞ , ta có

$$X[(i + |Y| - 1) \bmod |X| + 1] = Y[i].$$

Suy ra

$$X[i] = X[(i + |Y| - 1) \bmod |X| + 1].$$

Vì $\gcd(|X|, |Y|) = 1$, mọi ký tự của X đều bằng nhau. Từ $X[(i-1) \bmod |X| + 1] = Y[i]$ lại suy ra mọi ký tự của Y cũng đều bằng nhau, nên $X^\infty = Y^\infty$. $\square$

Nếu $\gcd(|X|, |Y|) = d$, lấy riêng các vị trí đồng dư modulo d trong X và Y để xét; dùng cách tương tự như trên có thể chứng minh chúng đều bằng nhau. Như vậy cũng suy ra $X^\infty = Y^\infty$. $\square$

Định nghĩa $T = f(c)^\infty[1 \dots 2N]$. Khi đó $g(i, c)$ có thể là $h(T[1 \dots i])$ hoặc $h(T[1 \dots i]) - 1$. Chỉ cần so sánh thứ tự từ điển của

$$T[1 \dots i]^\infty[1 \dots 2N]$$

hoặc

$$f(h(T[1 \dots i]) - 1)^\infty[1 \dots 2N]$$

với T , là có thể nhận được đáp án trong độ phức tạp $O(N)$.

Chú ý rằng với $i = N$, nếu giá trị $g(i, c)$ tìm được lớn hơn n , cần gán $g(i, c) = n$.

Độ phức tạp để tìm mọi $g(i, c)$ là $O(N^2)$.

5.5.2 Thống kê đóng góp của $f(c)$

Độ dài tiền tố bị lấy của $f(c)$ là

$$k - \sum_{i=1}^N i \times (g(i, c) - 2^{i-1} + 1).$$

Dùng tính toán số lớn cho biểu thức trên sẽ thu được độ dài l của phần bị lấy từ $f(c)$; sau đó chỉ cần tìm trong $f(c)[1 \dots l]$ có bao nhiêu ký tự 1.

5.5.3 Thống kê đóng góp của $f(P_1), f(P_2), \dots, f(P_{Q_c-1})$

Với mỗi $1 \leq i \leq N$, cần cộng vào đáp án số lượng ký tự 1 trong mọi số có độ dài i thỏa điều kiện, tức các số

$$2^{i-1}, 2^{i-1} + 1, \dots, g(i, c).$$

Giả sử $g(i, c) = a_1 a_2 \dots a_i$. Liệt kê mọi $2 \leq j \leq i$; nếu $a_j = 1$, thống kê đóng góp của mọi số trong khoảng

$$[\overline{a_1 a_2 \dots a_{j-1} 0} \times 2^{i-j}, \overline{a_1 a_2 \dots a_{j-1} 1} \times 2^{i-j}).$$

Như vậy tổng cộng đã thống kê đóng góp của các số trong khoảng $[2^{i-1}, g(i, c))$.

Đối với mọi số trong khoảng

$$[\overline{a_1 a_2 \dots a_{j-1} 0} \times 2^{i-j}, \overline{a_1 a_2 \dots a_{j-1} 1} \times 2^{i-j}),$$

đóng góp có thể được xử lý thành hai phần.

Với đóng góp của j bit cao nhất, vì j bit cao nhất trong khoảng này đều giống nhau, giả sử j bit cao nhất có tổng cộng x bit 0, thì đáp án cần cộng thêm $x \times 2^{i-j}$.

Với đóng góp của $i - j$ bit còn lại, có thể xem là số lượng bit 1 trong $[0, 2^{i-j})$. Với mọi $1 \leq k \leq i - j$, số lượng số có bit thứ k bằng 1 là 2^{i-j-1} . Vì vậy đáp án cần cộng thêm

$$(i - j) \times 2^{i-j-1}.$$

Dùng các bước trên để thống kê đóng góp cho một i cần $O(N)$ lần cộng $y \times 2^i$ vào đáp án, trong đó y là một số nguyên dương khá nhỏ. Có thể dùng số lớn với cơ số là lũy thừa của 2 để thực hiện việc này trong độ phức tạp xấp xỉ $O(N)$.

Vì cần thống kê đóng góp cho mọi $1 \leq i \leq N$, phần này có thể giải quyết trong thời gian xấp xỉ $O(N^2)$.

Quy trình trên cho ta cách giải hiệu quả sau khi đã tìm được c .

5.6 Phần thứ nhất

Thuật toán mô tả dưới đây chỉ cung cấp cách giải cho phần thứ nhất; cách giải cho phần thứ hai đã được xử lý trong phần trên.

5.6.1 Thuật toán ba

Liệt kê độ dài của c , sau đó tìm kiếm nhị phân giá trị của c . Với một giá trị mid khi tìm kiếm nhị phân, dùng phương pháp trên để tìm

$$g(1, mid), g(2, mid), \dots, g(N, mid),$$

rồi tính tổng độ dài

$$len = \sum_{i=1}^N i \times (g(i, mid) - 2^{i-1} + 1).$$

So sánh len với k để quyết định giữ nửa nào của khoảng tìm kiếm. Cuối cùng, nếu thỏa $0 \leq k - len < |f(c)|$, thì điều này cho thấy độ dài đang được liệt kê đúng là độ dài của c .

Độ phức tạp là $O(N^4)$.

Điểm kỳ vọng: 49 điểm.

5.6.2 Thuật toán bốn

Bổ đề 5. Với mọi $2^{N-2} \leq i < 2^{N-1} - 1$, có

$$Q_{i+1} - Q_i \leq 2N + 2.$$

Chứng minh. Với mọi $1 \leq j < N$, các số có độ dài j và nằm giữa i và $i + 1$ chỉ có thể là hai số:

$$f(i)[1 \dots j] \quad \text{và} \quad f(i+1)[1 \dots j].$$

Các số có độ dài N và nằm giữa i và $i + 1$ chỉ có thể có bốn số:

$$f(i) + "0", \quad f(i) + "1", \quad f(i+1) + "0", \quad f(i+1) + "1".$$

Vậy số lượng số nằm giữa i và $i + 1$ chỉ có thể là

$$2(N - 1) + 4 = 2N + 2.$$

□

Bổ đề 6.

$$n - Q_{2^{N-1}-1} \leq 1.$$

Chứng minh. Các số đứng sau $2^{N-1} - 1$ nhất định chỉ gồm toàn ký tự 1, và có độ dài lớn hơn $N - 1$.

Nếu $n = 2^N - 1$, thì $n - Q_{2^{N-1}-1} = 1$; ngược lại $n - Q_{2^{N-1}-1} = 0$. □

Bổ đề 7. Nếu $g(N - 1, c)$ tồn tại, thì

$$Q_c - Q_{g(N-1,c)} \leq 2N + 2.$$

Chứng minh. Từ Bổ đề 5 và Bổ đề 6, khoảng cách giữa mọi số có độ dài $N - 1$ không vượt quá $2N + 2$, và sau số cuối cùng có độ dài $N - 1$ nhiều nhất chỉ còn một số. Vì vậy với một số bất kỳ, nếu tìm được số có độ dài $N - 1$ lớn nhất đứng trước nó, thì khoảng cách giữa hai số đó không vượt quá $2N + 2$. □

Bổ đề 8.

$$Q_{2^{N-2}} = 1.$$

Chứng minh. Dễ thấy số duy nhất đứng trước 2^{N-2} là 2^{N-1} . □

Theo kết luận của Bổ đề 7 và Bổ đề 8, nếu trước c (không tính c) không có số nào có độ dài $N - 1$, thì c chỉ có thể là 2^{N-1} hoặc 2^{N-2} .

Ngược lại, có thể dùng phương pháp tìm kiếm nhị phân của thuật toán ba để tìm $g(N - 1, c)$. Giả sử $g(N - 1, c) = x$; tiếp theo cần tìm $2N + 2$ số đứng sau x , đồng thời duy trì

$$len = \sum_{i=1}^N i \times (g(i, x) - 2^{i-1} + 1).$$

Để tìm $2N + 2$ số đứng sau x , có thể duy trì $g(i, x)$ với mọi $1 \leq i \leq N$, rồi chèn

$$f(g(i, x) + 1)^\infty[1 \dots 2N]$$

vào một trie. Mỗi thao tác có thể tìm trong trie số có thứ tự từ điển nhỏ nhất trong $O(N)$ và xem nó là số tiếp theo; đồng thời cần cộng độ dài của số này vào len . Nếu sau một thao tác phát hiện $len > k$, thì số hiện tại chính là c .

Phần này có độ phức tạp $O(N^2)$.

Do đã bỏ được bước liệt kê độ dài, độ phức tạp của phần tìm kiếm nhị phân giảm xuống $O(N^3)$.

Điểm kỳ vọng: 72 điểm.

5.6.3 Thuật toán năm

Nhận thấy nút thắt của thuật toán bốn là tìm kiếm nhị phân, còn bước sau khi tìm kiếm nhị phân có vẻ khó tối ưu. Vì vậy có thể thử thay tìm kiếm nhị phân bằng cách xác định từng bit, duy trì một số

$$x = b_1 b_2 \dots b_{N-1}$$

và

$$len = \sum_{i=1}^N i \times (g(i, x) - 2^{i-1} + 1).$$

Ban đầu

$$b_1 = b_2 = \dots = b_{N-1} = 0.$$

Sau đó liệt kê theo thứ tự $b_1, b_2, \dots, b_{N-1}$ từ bit cao tới bit thấp; mỗi lần thử đặt b_i thành 1. Nếu sau khi đặt thành 1 phát hiện $len > k$, đặt lại b_i thành 0. Cuối cùng, x thu được chính là $g(N-1, c)$.

Ở đây cần hỗ trợ sửa một bit của x , đồng thời nhanh chóng duy trì giá trị len .

Đóng góp của $g(N, x)$ có thể được xử lý riêng trong thời gian $O(N)$.

Với $1 \leq i < N$, ta có

$$g(i, x) = h(f(x)[1 \dots i])$$

hoặc

$$g(i, x) = h(f(x)[1 \dots i]) - 1.$$

Định nghĩa dãy

$$d_1, d_2, \dots, d_{N-1} \quad (0 \leq d_i \leq 1)$$

thỏa

$$g(i, x) = h(f(x)[1 \dots i]) - d_i.$$

Khi đó ảnh hưởng của mỗi lần sửa là: $h(f(x)[1 \dots i])$ có thể bị sửa một bit, và d_i bị sửa. Ứng với len , thay đổi này làm len biến động một lượng

$$i \times (y \times 2^j + z) \quad (-1 \leq y, z \leq 1).$$

Nếu có cách duy trì nhanh các biến động của $h(f(x)[1 \dots i])$ và d_i , thì với mọi $1 \leq i < N$, có thể dùng số lớn với cơ số là lũy thừa của 2 để cộng tất cả các giá trị $i \times (y \times 2^j + z)$ trong độ phức tạp xấp xỉ $O(N)$.

Với việc sửa $h(f(x)[1 \dots i])$, chỉ cần quan sát xem thao tác sửa lần này có ảnh hưởng tới i bit đầu của x hay không. Nếu có, thực hiện sửa đổi tương ứng.

Với việc sửa d_i , định nghĩa

$$r_j = [f(x)^\infty[j] = f(x)^\infty[j + i]].$$

Nếu r_j toàn là 1, thì

$$d_i = [i = N - 1].$$

Ngược lại, giả sử vị trí đầu tiên trong r_j bằng 0 là l , khi đó

$$d_i = [f(x)^\infty[l] > f(x)^\infty[l+i]].$$

Theo Định lý 2, chỉ cần duy trì $2(N-1)$ phần tử đầu của r . Khi sửa một bit của x , trong $f(x)^\infty$ chỉ cần sửa 2 vị trí tương ứng, và trong r chỉ cần sửa 4 vị trí tương ứng. Sau đó bài toán trở thành sửa một phần tử của r , và nhanh chóng trả lời vị trí 0 đầu tiên của r .

Ở đây có thể dùng thuật toán chia khối. Trong phạm vi dữ liệu của bài này, $2(N-1) \leq 4000$, nên đặt kích thước khối là 64 và dùng một unsigned long long để duy trì thông tin của mỗi khối. Số lượng khối cũng không vượt quá 64, nên có thể dùng một unsigned long long để duy trì khối nào còn tồn tại 0. Khi truy vấn, trước hết dùng phép toán bit để tìm khối đầu tiên còn tồn tại 0, rồi trong khối này tìm vị trí 0 đầu tiên.

Như vậy có thể duy trì từng d_i trong độ phức tạp xấp xỉ $O(1)$, và việc sửa một bit của x cũng đạt độ phức tạp xấp xỉ $O(N)$.

Trong quá trình xác định từng bit, cần thực hiện $O(N)$ lần sửa. Vì vậy có thể tìm $g(N-1, c)$ trong độ phức tạp xấp xỉ $O(N^2)$, và giải quyết bài toán.

Điểm kỳ vọng: 100 điểm.

5.7 Tổng kết

Thuật toán cuối cùng của bài này không dùng bất kỳ kiến thức phức tạp nào. Nhìn bề ngoài đây là một bài số lớn đơn giản, nhưng nó kiểm tra năng lực tư duy, năng lực phân tích vấn đề, và năng lực viết mã của thí sinh. Thuật toán cuối cùng thay tìm kiếm nhị phân bằng xác định từng bit, kết hợp với nội dung riêng của bài để tối ưu; điều này có thể rèn luyện rất tốt tính mở rộng và tính sáng tạo trong tư duy của thí sinh. Độ khó cài đặt của bài này cũng khá cao, có thể kiểm tra nền tảng lập trình cơ bản của thí sinh.

Người ra đề hy vọng dùng bài này để gợi mở cho mọi người, mong rằng mọi người có thể tạo ra những bài hay chỉ dùng kiến thức đơn giản nhưng vẫn đạt độ khó tương đối cao và độ phân hóa tương đối tốt.

Tài liệu tham khảo

1. Liu Rujia. *Sách kinh điển nhập môn thi thuật toán*. Nhà xuất bản Đại học Thanh Hoa, 2009.
2. Cormen T. H., Leiserson C. E., Rivest R. L., và các tác giả khác. *Introduction to Algorithms*. MIT Press, 2009.

6 Báo cáo ra đề *Khối liên thông nhỏ nhất*

Pan Junyue, Trường Trung học số 2 Hàng Châu, tỉnh Chiết Giang

Tóm tắt

Bài viết này giới thiệu một bài tương tác do tác giả ra trong một kỳ thi thử liên trường nội bộ. Bài có nhiều cách giải, đòi hỏi thí sinh hiểu khá sâu cấu trúc của cây và biết vận dụng linh hoạt các tính chất của cây. Đây là một bài tốt để kiểm tra mức độ hiểu biết của thí sinh về cấu trúc dữ liệu cây.

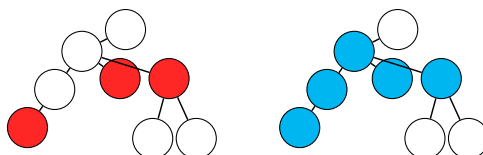
6.1 Ý chính bài toán

6.1.1 Mô tả bài toán

Đây là một bài tương tác.

Với một cây T , ta định nghĩa khối liên thông nhỏ nhất của một tập đỉnh trên cây là khối liên thông trên cây nhỏ nhất chứa tất cả các đỉnh trong tập đó.

Chẳng hạn trong hình dưới, tập đỉnh màu xanh là khối liên thông nhỏ nhất của tập đỉnh màu đỏ.



Biết kích thước n của một cây nào đó, bạn có thể thực hiện một số lần truy vấn. Mỗi lần truy vấn, bạn đưa ra một tập đỉnh S và một đỉnh x trên cây; thư viện tương tác sẽ trả về một giá trị boolean cho biết x có nằm trên khối liên thông nhỏ nhất của tập đỉnh S hay không.

Bạn cần xác định hình dạng của cây.

Bài đảm bảo cây được dùng đã hoàn toàn cố định trước khi bắt đầu tương tác, không được xây dựng động theo quá trình tương tác với chương trình của bạn.

6.1.2 Chi tiết cài đặt

Bạn không cần, và cũng không nên, cài đặt hàm chính. Bạn chỉ cần cài đặt hàm `work(n)`, trong đó n là số đỉnh của cây cần tìm. Bạn có thể gọi bốn hàm sau để tương tác với thư viện:

- `clear()`, biểu thị xóa rỗng tập đỉnh hiện tại S .
- `add(x)`, biểu thị thêm đỉnh số x vào tập đỉnh S .
- `query(x)`, biểu thị hỏi đỉnh x có nằm trên khối liên thông nhỏ nhất của tập đỉnh S hay không.
- `report(x, y)`, biểu thị xác nhận trong cây cần tìm có một cạnh (x, y) .

Khi chấm, thư viện tương tác sẽ gọi `work(n)` đúng một lần.

Ta chỉ giới hạn số lần thực hiện thao tác `query`.

6.1.3 Cách chấm điểm

Bài này có một gói kiểm thử, gồm nhiều điểm kiểm thử.

Với mỗi điểm kiểm thử, nếu chương trình của bạn có truy vấn hoặc giá trị trả về không hợp lệ, hoặc trả về hình dạng cây không đúng, thì bạn được 0 điểm ở điểm kiểm thử đó. Ngược lại, đặt `step` là số lần truy vấn của chương trình, điểm của bạn tại điểm kiểm thử đó được tính là

$$\min\left(\left\lfloor \frac{2.2 \times 10^6}{\text{step}} \right\rfloor, 100\right).$$

Điểm bạn nhận được cho bài này là giá trị nhỏ nhất trong điểm của tất cả các điểm kiểm thử. Có thể thấy, để đạt điểm tối đa, số lần truy vấn không được vượt quá 22000.

6.1.4 Giới hạn và quy ước

Với mọi dữ liệu, $n = 1000$.

Giới hạn thời gian: 5 giây.

Giới hạn bộ nhớ: 1024 MB.

6.2 Phân tích lời giải

6.2.1 Thuật toán một

Trong phần sau, ta gọi một thao tác “hỏi x có nằm trên khối liên thông nhỏ nhất của tập đỉnh S hay không” là $\text{query}(S, x)$.

Trước hết, ta chuyển đổi cách truy vấn đã cho. Hỏi x có nằm trên khối liên thông nhỏ nhất của tập đỉnh S hay không tương đương với hỏi trong S có tồn tại hai đỉnh u, v sao cho x nằm trên đường đi từ u đến v hay không.

Vì vậy, nếu tập được hỏi S có kích thước 2, ta có thể trực tiếp hỏi đỉnh x được truy vấn có nằm trên đường đi giữa hai đỉnh trong S hay không.

Ta có ngay một cách làm đa thức: với ba đỉnh bất kỳ u, v, w , dùng $\text{query}(\{u, v\}, w)$ để thu được quan hệ giữa chúng, từ đó xác định hình dạng của toàn bộ cây.

Nói cách khác, ta có:

Thuật toán một. Với mỗi cặp đỉnh (u, v) , ta thực hiện một lần $\text{query}(\{u, v\}, w)$ cho mỗi đỉnh còn lại w . Nếu đáp án của cả $n - 2$ truy vấn này đều là false , thì u và v nối trực tiếp với nhau. Nhờ đó có thể tìm tất cả các cặp đỉnh nối trực tiếp, rồi xác định hình dạng của toàn bộ cây.

Độ phức tạp thuật toán: $O(n^3)$.

6.2.2 Thuật toán hai

Có thể nhận thấy ta không cần lấy kết quả $\text{query}(\{u, v\}, w)$ cho mọi bộ ba đỉnh u, v, w . Trên thực tế, sau khi xem cây này như một cây có gốc, nếu ta biết được quan hệ “tổ tiên–hậu duệ” giữa mọi cặp đỉnh, thì cũng có thể xác định hình dạng cây. Đây cũng là một cách làm thường gặp để giải loại bài này.

Trong phần sau, ta đều xem cây là cây có gốc, đồng thời lấy đỉnh số 1 làm gốc.

Để biết giữa một cặp đỉnh u, v có quan hệ “tổ tiên–hậu duệ” hay không, chỉ cần hỏi $\text{query}(\{1, u\}, v)$.

Nói cách khác, ta có:

Thuật toán hai. Với mỗi cặp đỉnh (u, v) , ta dùng $\text{query}(\{1, u\}, v)$ để phán đoán v có phải tổ tiên của u hay không. Từ đó ta có thể thu được tập tổ tiên của mỗi đỉnh. Gọi tập tổ tiên của đỉnh i là Anc_i (đặc biệt, ta xem chính đỉnh i cũng là tổ tiên của i). Với một đỉnh x , cha f của nó chính là đỉnh thỏa $\text{Anc}_f \subset \text{Anc}_x$ và có $|\text{Anc}_f|$ lớn nhất. Xác định cha của mỗi đỉnh là xác định được hình dạng của cây.

Độ phức tạp thuật toán: $O(n^2)$.

6.2.3 Thuật toán ba

Chú ý rằng đến đây ta vẫn chỉ dùng các truy vấn với tập đỉnh có kích thước 2. Tiếp theo ta khai thác thêm cách dùng của dạng truy vấn được cho.

Vì có thể thu được quan hệ “tổ tiên–hậu duệ” giữa hai đỉnh, ta cũng có thể dùng cách tương tự để chỉ bằng một truy vấn biết giữa nhiều đỉnh có tồn tại quan hệ “tổ tiên–hậu duệ” hay không.

Với một tập đỉnh S và một đỉnh u , ta thực hiện $\text{query}(S \cup \{1\}, u)$. Nếu kết quả là true , điều đó nói rằng trong S có hậu duệ của u ; nếu không thì không có. Lý do là nếu trong S đồng thời có hai đỉnh, một đỉnh nằm trong cây con của u và một đỉnh nằm ngoài cây con của u , thì đường đi giữa chúng sẽ đi qua u , mà ở đây ta đã cung cấp sẵn một đỉnh ngoài cây con là đỉnh số 1. Tương tự, muốn tồn tại hai đỉnh sao cho đường đi tương ứng của chúng đi qua u , thì ít nhất một trong hai đỉnh đó phải nằm trong cây con của u . Vì vậy, “trong S có hậu duệ của u ” và “giá trị trả về của truy vấn là true ” là điều kiện cần và đủ của nhau.

Khai thác thêm một bước, sau khi xác nhận trong S có hậu duệ của u , ta có thể dùng cách “tìm kiếm nhị phân” để trực tiếp tìm một hậu duệ nào đó của u trong S . Cách làm cụ thể như sau:

- Chia tập đỉnh S thành hai tập đỉnh S_L, S_R , sao cho $S_L \cup S_R = S$, $S_L \cap S_R = \emptyset$, và $-1 \leq |S_L| - |S_R| \leq 1$.
- Thực hiện một truy vấn $\text{query}(S_L \cup \{1\}, u)$. Nếu giá trị trả về là true , sửa S thành S_L ; nếu không, sửa S thành S_R .
- Lặp lại hai bước trên cho tới khi $|S| = 1$. Khi đó đỉnh trong S là hậu duệ cần tìm.

Vì sau mỗi truy vấn, kích thước của S nhiều nhất trở thành $\lceil |S|/2 \rceil$, nên độ phức tạp của một thao tác “tìm một hậu duệ nào đó” là $O(\log |S|)$.

Khai thác xa hơn, ta có thể tìm tất cả các hậu duệ của u trong S bằng các bước sau:

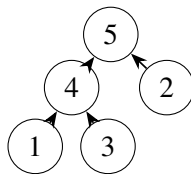
- Phán đoán trong S có hậu duệ của u hay không; nếu không thì thoát trực tiếp.
- Tìm một hậu duệ của u trong S .
- Loại hậu duệ vừa tìm khỏi S , rồi quay lại bước đầu tiên.

Giả sử số hậu duệ của u trong S là m , độ phức tạp của cách làm này là $O(m \log |S|)$.

Sau khi có các công cụ trên, ta thấy có thể chuyển hóa bài toán.

Nếu xem mỗi cạnh của cây có gốc là một cạnh có hướng từ con trở về cha, ta có thể xem cây có gốc như một đồ thị có hướng không chu trình, và định nghĩa một thứ tự topo của cây có gốc.

Ví dụ trong hình dưới, với một thứ tự topo nào đó của cây có gốc này, con số trên mỗi đỉnh thể hiện vị trí của nó trong thứ tự topo đó:



Ta nhận thấy nếu biết được một thứ tự topo của cây cần tìm, ta có thể trực tiếp xác định cây đó. Các bước cụ thể như sau:

- Gọi $a_1, \dots, a_n$ là số hiệu của đỉnh thứ i trong thứ tự topo của cây. Giá trị ban đầu của tập V là tập đỉnh của cây.
- Cho i quét từ 1 đến n . Ở mỗi bước, tìm tất cả hậu duệ của a_i trong V , rồi loại các hậu duệ này khỏi V .

Vì các hậu duệ không phải con của một đỉnh đã bị loại khỏi V tại cha tương ứng của chúng, nên các hậu duệ mà một đỉnh tìm được chính là tất cả các con của nó trong cây ban đầu.

Có thể chứng minh độ phức tạp của phần này là $O(n \log n)$, vì mỗi đỉnh chỉ bị tìm ra với vai trò hậu duệ đúng một lần, và mỗi lần tìm một hậu duệ đều tốn $O(\log n)$.

Từ đây, ta chuyển bài toán thành tìm một thứ tự topo của cây ban đầu. Tất cả các cách làm tiếp theo đều nhằm mục tiêu “tìm ra thứ tự topo”.

Xét thuật toán topo thường dùng: mỗi lần ta tìm một đỉnh có bậc vào bằng 0, đưa nó vào cuối thứ tự topo hiện tại, rồi xóa nó khỏi đồ thị ban đầu. Chuyển sang cây, tức là mỗi lần tìm một lá của cây ban đầu, rồi loại lá đó khỏi cây.

Ta gọi cách làm này là “bóc lá”.

Việc phán đoán một đỉnh u có phải lá hay không rất đơn giản. Gọi tập đỉnh của cây là V , chỉ cần hỏi $\text{query}(V - \{u\}, u)$.

Nói cách khác, ta có:

Thuật toán ba. Mỗi vòng, ta quét qua từng đỉnh của cây ban đầu và phán đoán nó có phải lá hay không. Nếu phải, đưa nó vào cuối thứ tự topo. Sau khi vòng đó kết thúc, loại tất cả các lá tìm được khỏi tập đỉnh của cây ban đầu. Cách phán đoán một đỉnh có phải lá hay không và cách làm sau khi tìm được thứ tự topo sẽ không được nhắc lại trong phần này và các phần sau.

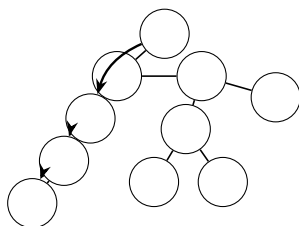
Độ phức tạp thuật toán: $O(n^2)$.

6.2.4 Thuật toán bốn

Trên thực tế, độ phức tạp của việc cố tìm một lá ở mỗi lần là quá cao; ta có thể thử tối ưu ở đây.

Ta lại dùng công cụ trước đó: mỗi lần liên tục tìm ngẫu nhiên một hậu duệ của đỉnh hiện tại trong tập đỉnh của cây rồi “nhảy” tới đó, cho tới khi tìm được một lá.

Hình dưới là một ví dụ cho quá trình “nhảy” của thuật toán:



Gọi siz_x là kích thước cây con của đỉnh x . Với mỗi đỉnh u , trong các hậu duệ của nó, số đỉnh có kích thước cây con vượt quá $\lceil \text{siz}_u / 2 \rceil$ nhiều nhất chiếm một nửa. Do đó kỳ vọng nhiều nhất 2 lần là có thể khiến kích thước cây con của đỉnh hiện tại giảm còn một nửa. Từ đây có thể chứng minh, nếu “nhảy” xuống dưới theo cách này, số lần “nhảy” kỳ vọng là $O(\log n)$. Mỗi lần tìm một hậu duệ cũng có độ phức tạp $O(\log n)$, nên độ phức tạp để tìm một lá là $O(\log^2 n)$.

Nói cách khác, ta có:

Thuật toán bốn. Đây là một thuật toán ngẫu nhiên. Ta bóc n lá để xác định thứ tự topo. Mỗi lần bắt đầu, đặt $u = 1$, rồi tìm ngẫu nhiên một hậu duệ v của đỉnh u , gán v cho u , cho tới khi u trở thành một lá.

Độ phức tạp thuật toán: $O(n \log^2 n)$.

6.2.5 Thuật toán năm

Thuật toán ba còn có một hướng tối ưu khác: tối ưu số vòng “bóc lá”.

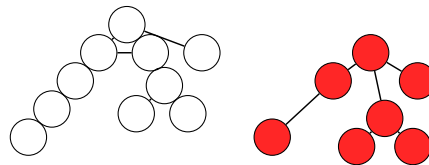
Có thể nhận thấy nếu mỗi vòng đều bóc tất cả các lá của cây ban đầu, trong trường hợp xấu nhất (khi cây ban đầu là một đường thẳng), cần bóc n vòng.

Ta phát hiện cây có một tính chất: nếu trong cây không có đỉnh bậc hai, thì số lá của cây ít nhất bằng một nửa tổng số đỉnh. Từ đây có thể nghĩ tự nhiên tới định nghĩa sau.

Định nghĩa cây ảo của một cây là cây thu được sau khi thực hiện các thao tác sau:

- Tìm một đỉnh bậc hai w ngoài gốc trong cây hiện tại, giả sử hai đỉnh kề với nó là u, v . Nếu không tìm được thì thoát.
- Xóa w và các cạnh nhận w làm đầu mút khỏi cây, rồi thêm cạnh (u, v) .
- Quay lại bước đầu tiên.

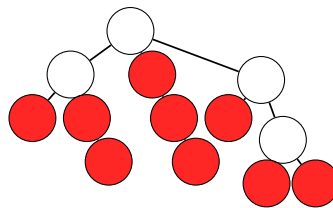
Ví dụ trong hình dưới, cây bên phải chính là cây ảo của cây bên trái.



Có thể nhận thấy, nếu ta khiến các lá của cây ảo của cây cần tìm đều bị bóc ở mỗi vòng, thì ta có thể bóc hết mọi đỉnh trong $O(\log n)$ vòng.

Muốn các lá của cây ảo ở một vòng nào đó đều bị bóc, những đỉnh ta cần bóc trong cây ban đầu là: các lá của cây ban đầu và chuỗi các đỉnh bậc hai nằm phía trên lá. Nói cách khác, đó là những đỉnh mà trong các hậu duệ chỉ có một hậu duệ là lá (đặc biệt, ta xem một đỉnh cũng là hậu duệ của chính nó).

Trong hình dưới, các đỉnh đỏ là những đỉnh ta cần bóc trong cây:



Đồng thời, việc tìm các đỉnh này cũng dễ. Chỉ cần trước hết tìm tập lá L , rồi với mỗi đỉnh, phán đoán nó có ít nhất 2 hậu duệ trong L hay không. Cách phán đoán là: trước hết tìm một hậu duệ v của đỉnh đó trong L , rồi xem trong $L - \{v\}$ có tồn tại hậu duệ của đỉnh đó hay không.

Sau khi tìm các đỉnh này, ta chia nhóm chúng theo hậu duệ lá tương ứng; trong mỗi nhóm còn phải sắp xếp một lần theo quan hệ “tổ tiên–hậu duệ”.

Như vậy ta hoàn thành công việc của một vòng với độ phức tạp $O(n \log n)$. Ta cần thực hiện tổng cộng $O(\log n)$ vòng, nên tổng độ phức tạp là $O(n \log^2 n)$. Cần đặc biệt chú ý rằng ở mỗi vòng, ta không làm số đỉnh của cây ban đầu giảm còn một nửa, nên không thể cho rằng độ phức tạp là $O(n \log n)$.

Nói cách khác, ta có:

Thuật toán năm. Mỗi vòng, tìm tất cả lá trong cây hiện tại. Sau đó, với mỗi đỉnh khác trên cây, tìm một hậu duệ lá của nó và phán đoán nó có ít nhất 2 hậu duệ lá hay không. Những đỉnh chỉ có một hậu duệ lá được chia nhóm theo hậu duệ lá đó; trong mỗi nhóm, sắp xếp các đỉnh, trong đó hai đỉnh u, v được so sánh bằng $\text{query}(\{1, u\}, v)$. Cuối cùng, bóc toàn bộ các đỉnh tìm được trong vòng này.

Độ phức tạp thuật toán: $O(n \log^2 n)$.

6.2.6 Thuật toán sáu

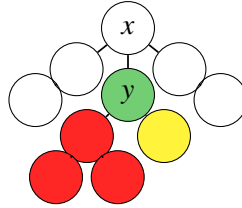
Không tiếp cận từ góc độ “bóc lá” nữa, ta xét cách làm thường gặp của loại bài này: chia để trị.

Ta đặt hàm $\text{solve}(x, S)$ biểu thị rằng đã xác định tập đỉnh trong cây con của x là S , và bây giờ cần sắp xếp các đỉnh trong cây con của x theo thứ tự topo.

Tìm ngẫu nhiên một hậu duệ y của x trong $S - \{x\}$. Ta cố xác định tập đỉnh S' tạo bởi các đỉnh trong cây con của y , để cây con của x có thể được chia thành hai phần, thuận tiện cho chia để trị.

Thực ra điều này không giống lắm với chia để trị theo trọng tâm truyền thống, nhưng ta vẫn có thể giải quyết bằng một thuật toán tất định có độ phức tạp tốt.

Để hiện thực ý tưởng này với độ phức tạp tốt hơn, trước hết ta giả sử các đỉnh trong cây ban đầu được tô các màu khác nhau theo thành phần liên thông chứa chúng sau khi xóa đỉnh y , như hình dưới:



Có thể thấy mục tiêu của ta là phán đoán mỗi đỉnh có phải màu trắng hay không. Trước hết, ta lấy các đỉnh này trừ y rồi sắp thành một hàng theo số hiệu đỉnh, có dạng như sau:



Gọi một đoạn liên tiếp cực đại gồm các đỉnh cùng màu là “một đoạn”. Bây giờ ta thử tìm vị trí các đường phân chia giữa các đoạn trong hàng đỉnh này.

Ta biết $\text{query}(A, y)$ trả về true khi và chỉ khi trong tập đỉnh A có hai đỉnh khác màu. Vì vậy ta có thể dùng một truy vấn để phán đoán một chuỗi đỉnh liên tiếp có cùng màu hay không. Khi đã xác định đầu trái của một đoạn, ta có thể dùng thao tác này để tìm kiếm nhị phân đầu phải của đoạn đó; quét từ trái sang phải là có thể xác định cách chia giữa các đoạn.

Cuối cùng, có thể dùng $\text{query}(\{1, u\}, y)$ để phán đoán đỉnh u có phải màu trắng hay không. Nhờ đó ta biết mỗi đoạn có phải màu trắng hay không, rồi tiếp tục chia để trị.

Xét chứng minh độ phức tạp của cách làm này.

Giả sử có m đỉnh được tô màu xuất hiện nhiều nhất. Độ phức tạp để xác định cách chia giữa các đoạn là $O((|S| - m) \log |S|)$. Nguyên nhân là mỗi lần xác định một đoạn đều tốn $O(\log |S|)$, còn số đoạn là $O(|S| - m)$: màu xuất hiện nhiều nhất bị chia thành nhiều nhất $|S| - m + 1$ đoạn, trong khi số đỉnh còn lại chỉ là $|S| - m$.

Nói cách khác, trong mỗi lần chia để trị, ta có thể xem các đỉnh có màu không phải màu xuất hiện nhiều nhất là những đỉnh đóng góp $O(\log |S|)$ vào độ phức tạp. Xét đóng góp của mỗi đỉnh:

- Nếu đỉnh đó là một đỉnh màu trắng, vì màu trắng không phải màu xuất hiện nhiều nhất, thành phần liên thông còn lại được tách ra lớn hơn thành phần liên thông màu trắng; kích thước của thành phần liên thông màu trắng không vượt quá một nửa kích thước thành phần liên thông ban đầu, nên tình huống này xảy ra không quá $O(\log n)$ lần.
- Nếu đỉnh đó là một đỉnh có màu, thì nếu thành phần liên thông màu trắng là lớn nhất, ta quay về tình huống thứ nhất. Ngược lại, gốc của thành phần liên thông chứa đỉnh đó là một con nhẹ của y . Lại vì mỗi đỉnh được chọn làm y nhiều nhất một lần, tình huống này cũng xảy ra không quá $O(\log n)$ lần.

Từ đó, ta chứng minh được tổng độ phức tạp là $O(n \log^2 n)$.

Nói cách khác, ta có:

Thuật toán sáu. Khi xử lý cây con của x , giả sử các đỉnh trong cây con lần lượt là $a_1, \dots, a_{\text{cnt}}$ (ở đây $\text{cnt} = |S|$). Tìm ngẫu nhiên một hậu duệ y của x , và giả định cây được tô màu theo đó. Ban đầu đặt $l = 1$; mỗi lần tìm kiếm nhị phân để tìm r lớn nhất thỏa $l \leq r \leq \text{cnt}$ và $a_l, \dots, a_r$ cùng màu, rồi đặt $l = r + 1$, cho tới khi $l > \text{cnt}$. Sau khi xác định cách chia giữa các đoạn, với mỗi giá trị l từng xuất hiện trước đó, hỏi $\text{query}(\{1, a_l\}, y)$ để phán đoán màu của đoạn. Cuối cùng chia để trị theo màu.

Độ phức tạp thuật toán: $O(n \log^2 n)$.

6.2.7 Thuật toán bảy

Thực ra, dùng tư tưởng tương tự chia để trị có thể thu được độ phức tạp tốt hơn.

Đặt hàm $\text{solve}(x, S)$ biểu thị yêu cầu tìm những đỉnh nào trong S nằm trong cây con của x , rồi sắp xếp chúng theo thứ tự topo. Nói cách khác, trước khi gọi $\text{solve}(x, S)$, ta chưa xác định những đỉnh nào nằm trong cây con của x .

Quy trình thuật toán như sau:

- Tìm một hậu duệ y của x trong $S - \{x\}$. Nếu không tồn tại hậu duệ như vậy thì thoát.
- Thực hiện hàm $\text{solve}(y, S)$, sau đó xóa khỏi S các đỉnh nằm trong cây con của y , và thêm các đỉnh trong cây con của y vào cuối thứ tự topo theo thứ tự topo của chúng.
- Quay lại bước đầu tiên.

Ban đầu chỉ cần gọi $\text{solve}(1, \{1, \dots, n\})$. Vì trong thuật toán này mỗi đỉnh chỉ bị tìm ra với vai trò y đúng một lần, tổng độ phức tạp là $O(n \log n)$.

Tư tưởng cốt lõi của thuật toán này là vứt bỏ những thông tin thực ra vô dụng, bóc cây con của x từng khối một cho tới khi chỉ còn x . Tuy quy trình rất đơn giản, độ khó tư duy để nghĩ ra nó lại không thấp.

Nói cách khác, ta có:

Thuật toán bảy. Khi xử lý cây con của x , liên tục tìm một hậu duệ y của x và chia để trị xuống y , rồi bóc phần cây con của y khỏi cây con của x ; lặp lại cho tới khi chỉ còn x .

Độ phức tạp thuật toán: $O(n \log n)$.

6.2.8 Thuật toán tám

Thực ra mọi cách làm ở trên đều xuất phát từ cấu trúc của cây, nhưng ta cũng có thể xét trực tiếp trên thứ tự topo.

Ta biết một điều kiện cần và đủ của thứ tự topo của một cây có gốc như vậy: mọi tổ tiên của mỗi đỉnh đều phải xuất hiện sau đỉnh đó. Nếu có thể lần lượt thêm các đỉnh vào thứ tự topo và luôn duy trì tính chất này, ta có thể giải quyết toàn bộ bài toán.

Ta đã có cách phán đoán trong tập đỉnh S có tồn tại hậu duệ của x hay không. Vì vậy chỉ cần tìm kiếm nhị phân một vị trí trong thứ tự topo hiện tại, sao cho sau vị trí đó không tồn tại hậu duệ của x và vị trí đó càng sớm càng tốt, thì ta có thể bảo đảm sau khi thêm đỉnh này, tất cả tổ tiên của nó vẫn nằm sau nó, và tất cả hậu duệ của nó vẫn nằm trước nó.

Vì mỗi lần thêm một đỉnh ta chỉ cần tìm kiếm nhị phân, tổng độ phức tạp là $O(n \log n)$.

Nói cách khác, ta có:

Thuật toán tám. Duyệt i từ 1 đến n . Giả sử trước khi thêm đỉnh số i vào thứ tự topo, thứ tự topo là $a_1, \dots, a_{i-1}$. Tìm kiếm nhị phân một vị trí p sao cho giá trị trả về của $\text{query}(\{1\} \cup \{a_p, \dots, a_{i-1}\}, i)$ là

false và p nhỏ nhất có thể. Chèn i vào trước vị trí thứ p , tức là sửa thứ tự topo thành

$$a_1, \dots, a_{p-1}, i, a_p, \dots, a_{i-1}.$$

Cuối cùng dùng lại cách làm sau khi đã có thứ tự topo.

Độ phức tạp thuật toán: $O(n \log n)$.

6.3 Tổng kết

Đề bài này ngắn gọn và tinh tế; kiến thức cần dùng chỉ là một số tính chất của cây, nhưng bài kiểm tra tổng hợp năng lực chế tạo công cụ và chuyển hóa vấn đề của thí sinh, đồng thời lời giải dùng nhiều kỹ thuật chia để trị. Bài không yêu cầu cao về năng lực viết mã, nhưng có yêu cầu nhất định về tư duy.

Để giải bài này, cần chuyển bài toán thành “tìm một thứ tự topo của cây ban đầu”. Trong quá trình chuyển hóa, bài nhấn mạnh tư tưởng “đóng gói dạng truy vấn được cho thành nhiều loại công cụ”. Sau khi chuyển thành “tìm thứ tự topo”, bài có thể được tiếp cận từ nhiều góc độ như “chia để trị theo đỉnh”, “bóc lá”, “duy trì trực tiếp thứ tự topo”, và đều có thể thu được các thuật toán khá tốt và khá thú vị. Chẳng hạn, ta khéo léo dùng tư tưởng “tô màu đỉnh và chia thành các đoạn” để thu được thuật toán sáu, đồng thời dùng nhiều tính chất của cây để chứng minh độ phức tạp của thuật toán sáu; một ví dụ khác là ta dùng tư tưởng “cây không có đỉnh bậc hai thì ít nhất một nửa số đỉnh là lá” để thu được thuật toán năm.

Người ra đề cho rằng một bài không cần kiến thức cao sâu nhưng vẫn có thể kiểm tra sâu mức độ hiểu biết của thí sinh về tính chất của cây như thế này xứng đáng là một bài hay, và cũng hy vọng bài này có thể gợi mở, để thấy thêm nhiều bài thú vị hơn.

6.4 Lời cảm ơn

Cảm ơn China Computer Federation đã cung cấp nền tảng học tập và trao đổi.

Cảm ơn huấn luyện viên đội tuyển quốc gia Gao Wenyuan vì sự hướng dẫn.

Cảm ơn thầy Li Jian của Trường Trung học số 2 Hàng Châu vì sự quan tâm và chỉ dẫn.

Cảm ơn các bạn Zhou Xin, Fang Youle, Fang Tangqi và những người khác đã thảo luận thuật toán của bài này với tôi, giúp kiểm tra đề.

Cảm ơn cha mẹ vì sự quan tâm và ủng hộ.

Tài liệu tham khảo

1. Codeforces 1129E, *Legendary Tree*.